

ZLang HDL

Language Reference

Community Release

v0.1.0a24

Installation, language, simulation, RTL generation, and verification

Apache-2.0 Community distribution

Contents

1 Installation, WSL2, and external tools **3**
1.1 Recommended installation with uv . . . 3
1.2 Conventional venv and pip alternative . . . 3
1.3 Windows through WSL2 . . . 4
1.4 Check and run ZLang . . . 4
1.5 Choose which external tools you need . . . 4
1.6 Recommended: OSS CAD Suite . . . 5
1.7 Alternative: install components separately . . . 5
1.8 Verify the installation . . . 6
1.9 Troubleshoot formal runs . . . 6

2 Native simulation **6**
2.1 Four-state initialization and diagnostics . . . 7
2.2 FST/VCD waveforms and differential RTL comparison . . . 7

3 Getting started **8**
3.1 Install for development . . . 8
3.2 A first module . . . 8
3.3 Check named verification goals . . . 9
3.4 Source files . . . 9
3.5 Reading common notation . . . 10
3.6 Where to continue . . . 10

4 Types, numerics, and packed representation **10**
4.1 Characters, fixed strings, and tuples . . . 10
4.2 Nominal enums . . . 11
4.3 Nominal tagged unions . . . 12
4.4 Scalar families . . . 12
4.5 Integer width rules . . . 12
4.6 Fixed point . . . 13
4.7 Vectors and structs . . . 14
4.8 Slicing, concatenation, and representation . . . 15
4.9 Current boundaries . . . 17

5 Expressions, functions, and generics **17**
5.1 Operators and selection . . . 17
5.2 Aggregate value ergonomics . . . 18
5.3 Pure functions . . . 18
5.4 Compile-time functions and selection . . . 19
5.5 Functional datapath . . . 20
5.6 Typed bit and collection composition . . . 21
5.7 Generic structs, functions, and operators . . . 22

6 Sequential state, rules, pipelines, and storage **22**
6.1 Clock and reset . . . 23
6.2 Registers and next state . . . 23
6.3 Delays and fixed pipelines . . . 25
6.4 Exact module timing contracts . . . 26
6.5 Guarded atomic actions . . . 26
6.6 FIFOs . . . 28
6.7 Writable memories . . . 28
6.8 Initialized synchronous ROMs . . . 32
6.9 Stateful hierarchy . . . 33

7 Physical clocks and resets **33**
7.1 Recommended asynchronous reset . . . 33
7.2 Hierarchy inheritance . . . 34
7.3 Boundaries . . . 34

8 Hierarchy, protocols, and explicit CDC **35**
8.1 Modules and instances . . . 35
8.2 Clock and reset boundary . . . 36
8.3 Public top ABI . . . 37
8.4 Ready/valid . . . 37
8.5 Credit and request/response . . . 37
8.6 Aggregate protocols and the standard library . . . 38
8.7 Packets, virtual channels, and arbitration . . . 38
8.8 CSR blocks . . . 38
8.9 Clock-domain crossings . . . 40

9 Named module interfaces **40**
9.1 Exact conformance . . . 41
9.2 Aggregate protocol signatures . . . 41
9.3 Current boundaries . . . 42

10 Tagged unions **42**
10.1 Type and layout . . . 42
10.2 Construction and matching . . . 42
10.3 Deliberate boundaries . . . 43

11 Parameterized aggregate protocols **43**

12 Standard buses **43**
12.1 Bounded ZTPU AXI burst subset . . . 45
12.2 AXI4-Lite, APB, and RegBus . . . 45

13 Standard library **45**
13.1 Fixed-point math . . . 47
13.2 Complex values, streams, storage, and coding . . . 47

14 Projects and dependencies **48**
14.1 Project manifest . . . 49
14.2 Lock update . . . 49
14.3 Offline compilation . . . 50
14.4 Deliberate boundaries . . . 50

15 Implementation intent, frequency, and II **50**
15.1 `intent` clauses . . . 51
15.2 Frequency control and synthesis trials . . . 51
15.3 Ready/valid contracts and II . . . 52

16 Formal verification **54**

17 Implementation profiles **54**

18 Platform constraints **55**
18.1 Clock constraints . . . 55

19 Generate SystemVerilog **55**

20 Known limitations **56**
20.1 Language and RTL boundaries . . . 56
20.2 Verification boundaries . . . 56

A ZLang HDL concise source-authoring reference **57**
A.1 Non-negotiable rules . . . 57
A.2 Smallest valid modules . . . 57
A.3 Types, literals, and exact conversions . . . 58
A.4 Aggregates and functions . . . 58
A.5 State, hierarchy, and protocols . . . 58
A.6 Functional datapath and implementation intent . . . 59
A.7 Verification . . . 59

A.8 Commands and completion check 59

This guide explains how to install ZLang HDL, write and organize hardware, simulate designs, generate SystemVerilog, and run formal checks. It focuses on source-language behavior and commands.

Direct SystemVerilog (Direct-SV) is the production RTL output. Unsupported language combinations fail with a diagnostic instead of producing partial or guessed hardware. The companion quick reference collects frequently used syntax, operators, conversions, and commands for daily authoring. The PDF edition appends that quick reference to this guide.

1 Installation, WSL2, and external tools

ZLang HDL supports CPython `>=3.12,<3.13`. The source compiler is release-tested on Linux x86-64. Native-simulation wheels are supplied for Linux x86-64 (including WSL2). macOS native wheels are not part of this release. You do not need to find a distribution package for that exact Python runtime: the recommended `uv` workflow can download and manage it independently of the system Python. Parsing, semantic checking and Direct-SV generation need only the Python package. Verilator, Yosys, SymbiYosys and a solver are external programs used only when their corresponding lint, synthesis or formal flow is requested.

The exact versions used by release acceptance are recorded in `release/status.json`. Other versions may work, but they are not evidence for the published release.

1.1 Recommended installation with uv

Install `uv` using its official platform instructions. Confirm that the executable is visible in the same terminal that will run ZLang:

```
uv --version
```

For ordinary use, install the wheel downloaded from the matching GitHub release as an isolated command-line tool. `uv` provisions the requested interpreter if it is not already present:

```
uv tool install --python '>=3.12,<3.13' \
  --with /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
zlang lock --version
zlang verify --version
```

If `uv` reports that its tool directory is not on `PATH`, run `uv tool update-shell`, then start a new shell. `uv tool dir --bin` prints the command directory for manual PATH configuration. To replace an installation made from a downloaded wheel, repeat `uv tool install --force ...` with the new wheel.

For development from a Git checkout, keep the environment local to the repository:

```
git clone https://github.com/postoroniy/zlang-hdl.git
cd zlang-hdl
uv venv --python '>=3.12,<3.13'
uv pip install -e '.[test]'
source .venv/bin/activate
zlang --version
```

The commands below use `zlang` directly: it is on `PATH` after `uv tool install`, or after activating the checkout's virtual environment with `source .venv/bin/activate` in a Bash/Zsh terminal. Alternatively, in the checkout use `uv run --no-sync zlang ...` without activating it. Open a new terminal and activate the environment again when returning to the checkout.

1.2 Conventional venv and pip alternative

If a compatible Python is already available, use whatever executable name the host provides; the example deliberately avoids assuming a version-suffixed command:

```
python3 -c 'import sys; assert (3, 12) <= sys.version_info < (3, 13), sys.version'
```

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install \
  /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
```

For an editable development checkout, replace the two-wheel installation command with:

```
python -m pip install -e '.[test]'
```

This fallback needs a compatible interpreter and `venv` support from the host. Do not replace or upgrade the operating system's own Python merely to install ZLang; use `uv` or an isolated virtual environment.

1.3 Windows through WSL2

The current release is tested as a Linux application, not as a native Windows Python package. On Windows 10/11, install WSL2 with an Ubuntu distribution, open the Ubuntu shell, and perform the Linux installation entirely inside it. For example:

```
sudo apt-get update
sudo apt-get install -y curl git build-essential
```

Then install `uv` using its official Linux instructions and follow the `uv` steps above. Keep active projects under the WSL Linux filesystem, for example `~/src/zlang-hdl`, rather than `/mnt/c/...`; this generally gives much better compiler, Git, tests and EDA-tool filesystem performance.

Inside a development checkout, these forms are equivalent:

```
source .venv/bin/activate
zlang examples/add.zhl --check
uv run --no-sync zlang examples/add.zhl --check
```

`uv run --no-sync` uses the already-created project environment without changing dependencies. In scripts, either activate the environment first or use `uv run --no-sync`; do not depend on a user's interactive shell state.

1.4 Check and run ZLang

Check the installed commands:

```
zlang --version
zlang lock --version
zlang verify --version
```

Check a design and emit production Direct-SV without installing any external EDA program:

```
zlang examples/add.zhl --check
mkdir -p build
zlang examples/add.zhl --systemverilog build/Add.sv
```

1.5 Choose which external tools you need

Only a compatible Python runtime and the `zlang-hdl` package are mandatory for checking source and emitting SystemVerilog. `uv` is an optional installer: it can obtain that Python runtime, but ZLang does not need `uv` after installation. Install other tools only for the workflows you actually use:

Workflow	Additional requirement	If it is missing
`zlang SOURCE --check` and SystemVerilog emission	None beyond Python and `zlang-hdl`	Neither Verilator nor Yosys is needed; a missing compiler package or incompatible Python prevents `zlang` from starting.
`zlang sim` (native simulation)	Matching `zlang-native-sim` wheel for the host	Simulation fails explicitly; compiler checking and RTL emission remain available.
Strict RTL lint or Verilator execution	`verilator`	RTL can still be emitted; lint and Verilator simulation cannot run.
Generic synthesis	`yosys`	RTL can still be emitted; synthesis cannot run.
Formal BMC, proof, or cover	`yosys`, `sby`, `yosys-smtbmc`, and one selected solver executable	Checking and RTL emission still work; requested formal jobs cannot complete.
Icarus-based test or simulation flow	`iverilog` and `vvp`	Only that selected flow is unavailable.
`zlang lock update` with Git dependencies	`git` and access to the pinned revision during update	Path-only projects and locked offline compilation still work; fetching a Git dependency does not.

`yosys-smtbmc` is the Yosys SMT model-checking driver. It is the executable sometimes informally shortened to "BMC"; there is no ZLang dependency named `bmct`. The default recorded formal route invokes the external `z3` executable through this driver. Bundle replay may instead select another supported SMTBMC solver, such as `boolector`, `bitwuzla`, `yices`, or `cvc5`, with `zlang verify BUNDLE --solver NAME`. Installing only a Python solver module is not a substitute for putting the corresponding executable on `PATH`.

1.6 Recommended: OSS CAD Suite

The simplest reproducible route is the official YosysHQ OSS CAD Suite. Its release archive provides Yosys, SymbiYosys, solvers and related open-source EDA tools together. Download the archive for the host platform from the OSS CAD Suite releases, extract it, then either source its environment file for each shell:

```
source /absolute/path/to/oss-cad-suite/environment
```

or add its `bin` directory to `PATH`:

```
export PATH="/absolute/path/to/oss-cad-suite/bin:$PATH"
```

Use an absolute installation path. Do not copy individual binaries out of the suite because Yosys and SymbiYosys also use adjacent data and support files. See the upstream OSS CAD Suite release page linked above for supported archives and platform details.

Install Verilator separately when it is not supplied by the selected suite. A distribution package is convenient:

```
sudo apt-get update
sudo apt-get install verilator
```

Distribution versions vary. Follow the upstream Verilator installation guide when the release-tested version or a source build is required.

1.7 Alternative: install components separately

Component installation is useful for distribution packaging or when a pinned tool is maintained independently:

- install Yosys using the upstream Yosys build/install instructions;
- install SymbiYosys using the upstream SymbiYosys documentation;
- install the `z3` command from the official Z3 releases or a suitable distribution package;
- install Verilator using its official guide linked above;
- install Icarus Verilog from the host distribution when the selected test or simulation flow requires `iverilog` /`vvp`.

Keep all selected tool `bin` directories on the same `PATH` used to start ZLang, the terminal, CI job or VS Code. Mixed host/container installations are a common reason for a tool being visible interactively but unavailable to the compiler.

1.8 Verify the installation

```
verilator --version
yosys -V
sby --version
command -v yosys-smtbmc
z3 --version
boolector --version
bitwuzla --version
iverilog -V
```

The release-tested external-tool versions are recorded only in the release status under `eda_toolchain`; the supported Python range is recorded under `platform.python`. These are evidence for that release, not universal tool-version requirements. Other versions may work; validate them for the flow you intend to use. Finding an executable on `PATH` is not, by itself, evidence that a formal or synthesis result is valid.

Run a real formal smoke test:

```
zlang examples/verification/bounded_counter.zhl \
  --verify --verify-require proven \
  --formal-depth 16 --formal-timeout 45 \
  --verification-bundle build/verify/counter \
  --verification-work-dir build/verify-work/counter \
  --verification-report build/verify/counter-result.txt
```

Expected exit status is zero. The `capacity` assertion should be `proven`, the `reaches_capacity` cover should be `witnessed`, and `exceeds_capacity` should be `bounded_unreached`. A bounded cover miss is not a proof of unreachability. See the runnable formal examples for a deliberate counterexample, scoped assumptions, immutable bundle replay and ready/valid checking.

1.9 Troubleshoot formal runs

Formal execution is opt-in. `--check`, normal compilation and SystemVerilog emission do not invoke a solver. `bounded_pass` is bounded evidence and is never reported as `proven`; unavailable tools or bindings remain explicit `unknown`/`skipped` according to the requested policy.

If a tool is not found, run the version commands above in the same environment that starts ZLang. If a proof is `unknown`, inspect the retained report and solver logs under `--verification-work-dir`; increasing depth or timeout does not turn an unsupported binding or reset model into an executable property.

2 Native simulation

Native simulation runs a checked design without first generating SystemVerilog or starting an external HDL simulator. Use the Python API when a test needs direct input, state, and clock control:

```
from zlang import sim

program = sim.compile("examples/counter.zhl", top="Counter")
instance = program.create()
instance.tick("clk")
print(instance.get("y"))
instance.close()
```

For ordinary command-line use, choose either a bounded single-clock run or an explicit event schedule:

```
zlang sim examples/counter.zhl --top Counter \
  --clock clk --cycles 100 --json
```

```
zlang sim examples/multi_clock_stateful.zhl --top MultiClockStateful \
  --events events.jsonl --trace build/trace.fst
zlang sim examples/add.zhl --top Add --set a=200 --set b=55 \
  --compare-with iverilog --compare-artifacts build/sim-compare --json
```

Use `--set NAME=JSON` for ordinary typed binary inputs. Repeat it for multiple ports. `--cycles N --clock NAME` is a convenience for one-clock designs whose reset/input state is already suitable. Use `--events` for resettable designs, multiple clocks, coincident edges, or changing inputs. For example:

```
{"reset":{"rst":true},"edges":["clk"]}
{"reset":{"rst":false},"set":{"load":1,"value":37},"edges":["clk"]}
{"set":{"load":0},"edges":["clk"],"repeat":4}
```

`set` changes input values before the event. `reset` sets the named physical reset signal level. `edges` contains every clock edge committed atomically from one pre-edge snapshot; therefore coincident clocks belong in the same array. The reset spelling is the pin level, so an active-low reset is asserted with `false`. Synchronous reset takes effect on its domain edge, while supported asynchronous assertion takes effect in the event that changes the reset level.

Each non-empty `--events` JSONL line is one simulation event. An event may add `"repeat": N` to apply the same updates, resets and edges for `N` consecutive events; `N` must be a positive integer and the expanded schedule is bounded to 1,000,000 events. The CLI retains one parsed mapping and repeat count per JSONL record rather than copying its nested payload `N` times. This keeps long idle or backpressure intervals declarative without requiring a project-specific event generator.

2.1 Four-state initialization and diagnostics

Ordinary mode uses deterministic binary-zero startup for state without an initializer. `--logic-state` instead initializes every unreset register bit to `U`, meaning it has never been refreshed. A write derived from unresolved state produces `X`; exact controlling values still resolve (`U | 1 == 1`, `X & 0 == 0`). This is hardware-debug information, not electrical `Z`, drive strength, or the Verilog event-region model.

Packed four-state spellings are MSB-first strings containing `0`, `1`, `u`, or `x`. Use `--set-logic NAME=01ux` for an input and `--initial-reg NAME=01ux` to override one named register path before the first event. The width must match exactly. `--strict-uninitialized` implies logic-state execution and fails when a typed observation requires an unresolved `U` or `X`; it does not alter compilation or Direct-SV.

2.2 FST/VCD waveforms and differential RTL comparison

`--trace PATH.fst` writes the preferred compact native waveform; `--trace PATH.vcd` retains VCD interoperability with external and formal tools. The suffix selects the format. Other suffixes, including `.ghw`, are rejected. Physical clocks and resets are always present; repeat `--trace-signal NAME` to keep only selected data/state signals. The timeline uses two 1 ns ticks per simulation event and reflects the declared edge/reset contract. It is an event visualization, not an analog delay model. Standard FST and VCD both collapse unresolved provenance to `x`, so ZLang emits hidden `__zlang_meta/*_u_mask` companions that its query service combines back into distinct `U` and `X` values.

`--compare-with iverilog` and `--compare-with verilator` emit the normal direct SystemVerilog artifact, execute it in the selected external simulator, and compare every physical output after every event. A mismatch, X/Z value, missing sample, tool failure, or timeout fails closed. Comparison artifacts are retained only when an empty `--compare-artifacts` directory is supplied. Sequential RTL must receive the same explicit reset events needed by physical hardware; use `--events` when comparing a resettable design.

Exit status is nonzero for compilation/plan errors, native runtime failures, strict-U/X violations, verification/protocol failures, comparison mismatch, external tool failure, or timeout. Machine consumers should use `--json` and still treat the process status as authoritative. `--compare-artifacts` must name a new empty directory; without it, temporary comparison material is discarded.

One instance is not thread-safe; independent instances may run concurrently. `eval()` changes no architectural state. `edge_many()` computes all next-state values from one pre-edge snapshot and commits atomically. Synchronous resets act on their domain edge, while supported asynchronous assertion acts immediately. Values are exact-width and are never silently truncated.

The following are implementation limits of this compiler release's native simulator, not ZLang language semantics. Native simulation accepts at most:

- 32,768 compiled simulation operations;
- a cumulative packed-value work budget of 32,768 machine words;
- 16 MiB of compiled simulation data;
- packed values up to 8,192 bits;
- arithmetic operands up to 512 bits; and
- memory elements up to 8,192 bits.

Generated functional work is also bounded to 8 nested regions, 1,000,000 total iterations, and an estimated runtime-work budget of 8,000,000 operations. Compilation fails with a diagnostic when a limit is exceeded.

There is no silent fallback to an RTL simulator. Native simulation requires the matching Linux x86-64/WSL2 wheel described in the installation chapter; an unsupported design or missing runtime produces an error.

3 Getting started

The representative executable language tour is `examples/all_syntax.zhl`. It is a practical starting point rather than an exhaustive list of every legal composition. See Known limitations for deliberately unsupported forms.

Physical ZLang HDL source files use the canonical `.zhl` suffix and MIME type `text/x-zlang-hdl`. Logical imports remain extension-independent. The former `.zl` spelling is intentionally rejected so these sources cannot be confused with the unrelated language that already owns that extension.

3.1 Install for development

This alpha release requires CPython `>=3.12,<3.13`. The recommended `uv` workflow does not require a suitable Python interpreter to be installed first; `uv` can download and manage the verified runtime. From the repository root:

```
uv venv --python '>=3.12,<3.13'
uv pip install -e '.[test]'
```

If a compatible interpreter is already available, the complete installation guide also documents a conventional `venv`/pip path without relying on a version-specific executable name. Windows users should run the Linux flow inside WSL2 rather than installing the compiler into native Windows Python.

Checking source and generating SystemVerilog do not require an EDA installation. Verilator, Yosys, SymbiYosys, `yosys-smtbmc`, Z3 and Icarus are optional external programs for the corresponding lint, synthesis, formal and simulation flows. See Installing ZLang HDL and the external EDA toolchain for the recommended OSS CAD Suite route, separate component installation, release-tested versions and a real formal smoke test.

Check a source file without generating output files:

```
zlang examples/all_syntax.zhl --check
```

Without `--top`, `--check` validates every declared module. With `--top NAME`, it validates only that selected root and does not generate RTL or run formal checks.

3.2 A first module

```
module Add {
    in  a : u8
    in  b : u8
    out y : u9

    y = a + b
}
```

Hardware assignments are concurrent. Source order does not turn `=` into software-style sequencing. `=` drives a combinational value; `<-` schedules a register's next value at its clock edge.

Generate Direct-SV and lint it:

```
zlang examples/extended_add.zhl \
  --systemverilog build/ExtendedAdd.sv
verilator --lint-only --top-module ExtendedAdd build/ExtendedAdd.sv
```

An explicit artifact path keeps stdout empty. Use `--verbose` for success messages on stderr. A bare invocation emits the production Direct-SV artifact to stdout; use `--check` for semantic validation without emission.

3.3 Check named verification goals

Clocked modules can carry same-cycle safety and bounded-reachability goals:

```
assert count_within @ clk { count <= DEPTH }
cover reaches_full @ clk { count == DEPTH }
```

Run the applicable verification jobs and source goals:

```
zlang design.zhl --top Top --verify \
  --formal-jobs 4 \
  --verification-report build/verification.txt \
  --verification-work-dir build/verification-work
```

Or publish a hash-validated bundle and replay it without recompiling source:

```
zlang design.zhl --top Top \
  --verification-bundle build/verify
zlang verify build/verify --mode bmc --depth 20 \
  --work-dir build/verify-work
```

`bounded_pass` means only that no counterexample was found through the stated depth. A cover reports `witnessed` or `bounded_unreached`; a cover miss is not a safety failure. A `prove` request runs BMC first and starts proof only when every safety job is `bounded_pass`. Solver configurations, logs, and VCDs are retained in the external work directory; the immutable bundle is not modified. `--formal-jobs` runs independent jobs in parallel while keeping result order deterministic.

Each verification goal is routed against its own declared clock/reset pair. Goals in two supported synchronous domains can execute independently; an unsupported domain skips only goals in that domain. This does not add a cross-domain temporal property.

3.4 Source files

A file may contain imports, type aliases, structs, pure functions, operator overloads, rewrite declarations, target/resource declarations, and one or more modules:

```
import std.math.complex

type Byte = u8

struct Pair<type T> {
    left  : T
    right : T
}

fn widen(x : Byte) -> u9 { extend<9>(x) }

module Example {
    in  x : Byte
    out y : u9 = widen(x)
}
```

Select a top explicitly when a file contains several modules:

```
zlang design.zhl --top Example --check
```

Names are case-sensitive. There are no semicolons. Line comments use `//`. Non-nested block comments use `/* ... */`; both forms are lexical whitespace. Decimal, hexadecimal (`0x`), and binary (`0b`/`0B`) integer literals may contain valid underscore separators.

3.5 Reading common notation

Form	Meaning
`name = expression`	Drive a combinational value or resource control.
`state <- expression`	Schedule next-state at the active edge.
`source -> destination`	Connect typed endpoints or bind a command.
`key => expression`	Select a `switch`/choice arm.
`thing @ clk`	Associate a declaration with a clock domain.
`field @ 7:4`	Place a CSR field.
`object.field`	Select an aggregate, protocol, or resource member.
`values[index]`	Read a range-proven vector element; the selector may be runtime.
`raw[index]` for a packed scalar	Read one LSB-zero packed bit; a runtime selector needs a compiler-proven in-range unsigned interval.
`raw[offset +: WIDTH]`	Read a fixed-width LSB-based packed window; `WIDTH` is compile-time and the runtime offset must be provably in range.

ZLang requires exact canonical types at assignment boundaries. It never silently truncates, extends, performs a numeric signed/unsigned conversion, or rescales fixed-point values. Equal-width raw boundaries involving `bits<N>` or flat `vec<N,bit>` are the documented representation-only exception; use `bitcast<T>` explicitly when the boundary is not already typed.

3.6 Where to continue

- Types and numerics
- Expressions, functions, and generics
- Sequential logic, rules, and storage
- Hierarchy, protocols, and composition
- Formal verification
- SystemVerilog generation and CLI
- Standard library
- Reproducible projects and dependencies
- Named module interfaces
- Runnable examples

4 Types, numerics, and packed representation

ZLang types describe hardware representation exactly. Width, signedness, fixed-point scale, vector length, struct identity, and fixed-point overflow policy participate in typing and canonical identity.

4.1 Characters, fixed strings, and tuples

`char` is a source alias for canonical `u8`; it is intentionally not a separate nominal type. `string<N>` is a source alias for the corresponding fixed hardware collection and normalizes to canonical `vec<N,u8>`:

```
letter : char = 'A'
tag : string<4> = "OFDM"
joined = concat("802.", "11a")
```

Literals contain printable ASCII directly and may use `\0`, `\n`, `\r`, `\t`, `\\`, `\'`, `\"`, or an exact `\xNN` byte. Strings have no hidden terminator or length field and cannot be empty. They inherit vector indexing, equality, `length`, `generate`/`map`, homogeneous concatenation, pure functions, generic specialization, and typed compile-time constant parameters. Registers, FIFOs, synchronous memories, and ROMs store them with the same semantics as `vec<N,u8>`. When packed, the first character is the least-significant byte (`pack("AB") == 0x4241`). Unicode, interpolation, formatting, padding, and dynamic-length strings are not implicit operations.

Tuples are structural ordered aggregates. Tuple types use `(T,U)` and tuple values use the same positional spelling:

```
pair : (u8, bit) = (data, last)
payload = pair[0]
(next_data, next_last) = pair
```

Tuple arity is 2 through 8. Projection is a zero-based integer literal and flat destructuring introduces exhaustive immutable bindings. Nested tuple values are legal, but nested/wildcard patterns, runtime tuple selection, tuple update, arithmetic, and ordering are not. Exact `==`/`!=` is structural. Component zero occupies the least-significant packed region; `pack`/`bitcast` require every component to satisfy the ordinary non-enum packing rules.

`char` is indistinguishable from `u8` during overload resolution, and `string<N>` is indistinguishable from `vec<N,u8>`. Tuple identity, by contrast, is the recursive ordered sequence of component types.

4.2 Nominal enums

Enums give control state a nominal hardware type without exposing a numeric encoding in source:

```
enum State { Idle Header Payload Done }

reg state : State = State.Idle
active = state == State.Payload
```

Members use qualified names. Their declaration-order ordinals are encoded in `max(1, ceil_log2(member_count))` bits, so a one-member enum still occupies one bit. The encoding is a stable storage/backend ABI, but enum values are not integers: arithmetic, ordering, resizing, and cross-enum comparison are errors. Equality and inequality require the same nominal enum declaration.

When an external encoding is part of the hardware contract, declare an exact `bit`, `bits<W>`, or unsigned `uN`/`u<W>`/`uint<W>` representation. Omitted member values start at zero and otherwise continue from the preceding resolved value:

```
enum StatusCode : u8 {
    Idle
    Started = 10
    Waiting = 120
    Complete
}
```

This assigns `0`, `10`, `120`, and `121`. Every resolved code must be unique and fit the declared backing type; `enum Flag : bit { Clear Set }` is therefore limited to two values. Declaration order still controls exhaustive source selection, while the resolved code is the physical representation. The unbacked ordinal form above remains unchanged.

Raw boundaries use total, typed operations rather than enum casts:

```
raw     : bits<3> = enum_encode(rate)
valid   : bit = enum_valid<WifiRate>(raw)
decoded : WifiRate =
    enum_decode<WifiRate>(raw, WifiRate.Continue)
```

`enum_encode` is lossless and returns the enum's exact declared backing type (`bit`, `bits<W>`, or `uN`/`u<W>`/`uint<W>`); an unbacked ordinal enum retains its minimal `bits<W>` representation. `enum_valid<T>` and `enum_decode<T>` require that same exact representation. Decode returns its required same-enum fallback for every hole, so an invalid representation never becomes an invalid nominal value. These operations perform no resize.

Enum values remain nominal and cannot participate directly in arithmetic. An unsigned-backed enum deliberately exposes a numeric boundary through `enum_encode`, so `enum_encode(status) + 1` uses the normal widened unsigned arithmetic rules. A `bit`- or `bits<W>`-backed enum requires an explicit `bitcast<uW>` before arithmetic; there is no implicit raw-bits-to-integer cast.

Enums may appear in locals, outputs, registers/rules, structs, and vectors. A top-level enum input is rejected because an external invalid bit pattern would violate the nominal value invariant. General `unpack<Enum>` is

likewise deferred. Exhaustive selection is described under operators and selection.

4.3 Nominal tagged unions

A tagged union represents one of several named payload shapes without exposing raw tag arithmetic:

```
union Message {
    Idle
    Data { value : u8 }
    Error { code : bits<4> }
}

message : Message = Message.Data { value = input }
result : u8 = match message {
    Message.Idle => 0
    Message.Data { value } => value
    Message.Error { code } => extend<8>(code)
}
```

The nominal declaration identity is part of the type. The source-order tag is stored in the most-significant bits; fields occupy the maximum-width payload in source order and shorter payloads receive zero low-order padding. A fieldless constructor is written `Message.Idle`. Every pure `match` is exhaustive, names each variant once, and binds exactly the declared fields. See the complete bounded surface.

4.4 Scalar families

Spelling	Meaning
`bit`	A one-bit control value, distinct from integers and raw bits.
`uN`, `u<N>`, `uint<N>`	An `N`-bit unsigned integer.
`sN`, `sint<N>`	An `N`-bit two's-complement signed integer.
`bits<N>`	An `N`-bit raw vector without integer arithmetic.
`fixed<W,F>`	Signed fixed point, `W` total bits and `F` fractional bits, wrapping target narrowing.
`ufixed<W,F>`	Unsigned fixed point with wrapping target narrowing.
`fixed_sat<W,F>`	Signed fixed point with saturating target narrowing.
`ufixed_sat<W,F>`	Unsigned fixed point with saturating target narrowing.

Widths are positive compile-time integers. Fixed-point types additionally require `0 <= F < W`. The families are distinct: no implicit signed/unsigned, integer/raw-bit, or integer/fixed conversion is performed.

Aliases disappear during typing:

```
type Address = uint<32>
type Payload = bits<64>
```

Alias cycles, duplicate declarations, and redefinition of built-ins are errors.

4.5 Integer width rules

An unsuffixed integer literal has the smallest exact hardware type implied by its value. Non-negative literals are unsigned; syntactic negative literals are signed two's-complement values:

Literal	Inferred type
`0`, `1`	`u1`
`2`, `3`	`u2`
`255`	`u8`
`-1`	`s1`
`-2`	`s2`
`-3`, `-4`	`s3`

Literal	Inferred type
`-123`	`s8`

Decimal, hexadecimal, and binary spellings with the same value infer the same type; leading zeros do not request storage width. A direct literal at an explicitly typed assignment, field, argument, or return boundary instead uses that target type when the value fits exactly. An out-of-range literal is an error. Context never recursively resizes a compound expression, so `concat(0,1)` and `0 + 1` retain the types derived by their own operators.

Unary `-` applied to a non-literal remains ordinary typed arithmetic. It does not reinterpret an unsigned signal as a signed one.

Expression	Result
`uN + uM`	`u(max(N,M)+1)`
`sN + sM`	`s(max(N,M)+1)`
`uN - uM`	`u(max(N,M))`, modular underflow
`sN - sM`	`s(max(N,M)+1)`
same-family `N * M`	Width `N+M`
`&`, `|`, `^`	Matching family, widest operand width
shifts	The left operand's exact type and width
comparisons	`bit`

Addition and multiplication preserve carry bits. A left shift discards bits beyond its fixed result width. Signed right shift is arithmetic; unsigned and raw-bit right shift is logical.

Use explicit resizing:

```
wide = extend<17>(value)
low  = truncate<8>(wide)
```

`extend<N>` preserves signedness behavior and `truncate<N>` keeps low bits. Neither changes the type family.

At an explicitly typed output, local, register write, storage write, child binding, struct field, or declared function-return boundary, the width may be obtained from that boundary while the conversion remains explicit:

```
low : u8 = truncate(wide)
wide_again : u17 = extend(low)
```

The contextual and explicit-width forms normalize to the same typed resize. An inferred local, a nested arithmetic operand, or a mismatched type family has no such context and must keep `truncate<N>` or `extend<N>`.

4.6 Fixed point

Concrete spellings expose integer and fractional widths directly:

Spelling	Canonical type	Narrowing policy
`SF8.8`	`fixed<16,8>`	wrap
`SF_Sat8.8`	`fixed_sat<16,8>`	saturate
`UF8.8`	`ufixed<16,8>`	wrap
`UF_Sat8.8`	`ufixed_sat<16,8>`	saturate

For signed values the integer part includes the sign bit. `SF8.8` therefore represents `-128.0` through `127.99609375` in steps of `2^-8`.

Arithmetic is widened before any storage conversion. Fixed addition and subtraction require equal fractional widths; multiplication returns total width `W1+W2` and fractional width `F1+F2`. Target overflow policy applies only at an explicit or contextual storage conversion, not after every intermediate node.

Direct fixed-point literals must be exactly representable and in range, even for a saturating target. Loss is explicit:

```
y = quantize<SF_Sat8.8>(value) {
    round nearest_even
    overflow saturate
}
```

The rounding mode is mandatory; there is no implicit rounding default.

`round` mode	Discarded fractional part
`toward_zero`	Truncate toward zero.
`floor`	Round toward negative infinity.
`away_zero`	Round away from zero if any fractional part remains.
`nearest_even`	Round to nearest; on an exact tie choose the even stored integer.

Form	Target and overflow behavior
`quantize(value, MODE)`	Target comes from an unambiguous fixed-point assignment/return context; overflow defaults to that target type's policy.
`quantize(value) { round MODE overflow POLICY }`	Contextual target; both clauses are required.
`quantize<T>(value) { round MODE overflow POLICY }`	Explicit fixed-point target; both clauses are required.

`POLICY` is `wrap` (modular truncation to the target bit width) or `saturate` (clamp to the target's minimum/maximum). Ordinary `fixed`/`ufixed` targets default to `wrap`; `fixed_sat`/`ufixed_sat` targets default to `saturate` only in the short two-argument form. The block's explicit policy takes precedence. Conversion order is exact rescale, rounding, then overflow. `fixed_raw(0x0C22)` creates a target-context raw pattern; `fixed_to_raw` exposes the stored integer.

An exact fixed-point `dot(a,b)` retains the full accumulator. If the destination reduces fractional precision, give one post-accumulation rounding mode:

```
out y : SF_Sat8.8
y = dot(coefficients, samples, floor)
```

4.7 Vectors and structs

`vec<N,T>` has a compile-time length:

```
in samples : vec<4,u8>
```

An explicitly typed boundary can use a non-empty vector literal or contextual replication:

```
pair          : vec<2,u8> = [left, right]
zero_values   : vec<4,u8> = repeat(0)
zero_values_2 : vec<4,u8> = repeat<4>(0)
```

Every element must have the exact contextual element type. `repeat(value)` requires the destination vector to provide its length; `repeat<N>(value)` makes the length explicit. Both forms retain ordinary vector sequence order and lower to the existing typed generation representation.

Constant indexes are checked during elaboration. Runtime reads are accepted only when conservative range analysis proves every encoded value in range; ZLang does not silently mask or clamp. A rule may update one range-proven element of a one-dimensional `reg vec<N,T>` atomically. Nested paths, multiple dynamic writes to the same register action group, and runtime-selected instances remain unsupported; see Sequential logic, rules, and storage.

Structs are nominal products:

```
struct Packet {
    data : u64
```

```
    last : bit
}

packet = Packet { data = payload last }
```

Constructors are complete and may use same-name field punning. An immutable update rebuilds the same nominal struct and preserves fields not named in the update:

```
completed = packet with { last = 1 }
```

Unknown or duplicate fields and values of the wrong exact field type are rejected. This is a pure value expression, not mutation. Recursive layouts are still rejected. Generic structs are covered in Expressions, functions, and generics.

Structural `==` and `!=` are available for exact matching struct and vector types. Comparison is recursive over every field/element and returns `bit`; it does not perform conversion, resizing, or nominal coercion.

An immutable nominal struct can also be destructured exhaustively:

```
Packet { data, last } = packet
```

Every declared field must appear exactly once, under its declared name, and the new immutable bindings may not shadow another symbol. Destructuring lowers to ordinary typed field values; it does not add tuple patterns, partial patterns, renaming, mutation, or runtime matching.

4.8 Slicing, concatenation, and representation

An inclusive, compile-time bit slice returns raw bits:

```
in word : u16
out byte : bits<8>
byte = word[15:8]
```

Both `MSB` and `LSB` are compile-time integer expressions. The result is `bits<MSB-LSB+1>`. Reversed, negative, or out-of-range bounds are errors; slicing never resizes, reinterprets, or clamps its operand.

A `bits<N>` value also supports concise single-bit selection:

```
in raw : bits<24>
out lsb : bit = raw[0]
out msb : bit = raw[23]
```

Packed indices use conventional bit numbering: index zero is the least-significant bit, exactly as `[0:0]`. A runtime unsigned selector is legal only when its compiler-proven interval lies entirely within the packed width; there is no implicit modulo, clamp, or out-of-range hardware behavior. A fixed-width runtime window uses LSB-based `+:` spelling:

```
in raw    : bits<16>
in offset : u3
out byte  : bits<8>
byte = raw[offset +: 8]
```

`WIDTH` is a positive compile-time integer. The compiler requires `offset + WIDTH <= packed_width` for every possible offset. Vector element zero likewise occupies the least-significant packed element region.

`concat(a,b,...)` accepts at least two operands. When every operand is a vector with the exact same element type it returns one longer vector, preserving source and element order:

```
in left  : vec<2,u8>
in right : vec<3,u8>
out all  : vec<5,u8>
all = concat(left, right) // left[0], left[1], right[0], right[1], right[2]
```

Otherwise no operand may be a vector and all operands must be recursively bit-packable. The result is raw `bits<N>` and the first operand occupies the most-significant bits:

```
joined = concat(high, low) // high is the MSB portion
```

Each scalar operand is typed independently, before the result width is known. For example, `concat(0,1)` is `bits<2>` and `concat(3,0)` is `bits<3>`. Neither an assignment target nor a function caller may widen an operand or the completed concatenation implicitly.

Exact packed fill constants are available without manufacturing a numeric literal of the desired width:

```
clear_mask : bits<24> = zeros<24>
set_mask   : bits<24> = ones<24>
```

The width is a positive compile-time integer expression. Both forms produce exact `bits<N>` values and lower to ordinary constants; they are not vector replication. The separate `zero<x>`/`ones<x>` spellings inside an `equiv` declaration remain typed pattern constants rather than hardware expressions.

Vector/scalar mixtures and different vector element types are errors. Convert a vector explicitly with `bitcast<bits<W>>` when raw bit assembly is intended.

`reshape(value)` obtains a vector target shape from an explicitly typed context; `reshape<T>(value)` supplies it directly. Reshape recursively enumerates leaves outer-to-inner, then rebuilds the requested shape without packing them:

```
in matrix : vec<2,vec<4,u8>>
out flat  : vec<8,u8>
flat = reshape(matrix)
matrix_again = reshape<vec<2,vec<4,u8>>>(flat)
```

Source and target must be vectors with the same leaf count and exact same leaf type. Reshape has no endianness and performs no numerical conversion.

`bitcast<T>(value)` exposes or reconstructs the exact stored representation when source and target have equal packed width and both are recursively bit-packable and non-enum. It never resizes, sign-extends, truncates, rescales, rounds, or saturates:

```
raw    : bits<8> = bitcast<bits<8>>(signed_value)
signed : s8      = bitcast<s8>(raw)
```

`pack(value)` and `unpack<T>(raw)` remain supported low-level compatibility spellings and normalize to the same typed `Bitcast` operation. Signed integers and signed fixed-point values use their stored two's-complement pattern.

Aggregate layout is deterministic across simulation and generated RTL:

- struct field zero (declaration order) is at the MSB;
- vector element zero is at the LSB;
- tuple component `item0` is at the LSB;
- nested vectors and tuples apply the LSB-first rule recursively;
- nested structs retain declaration-order/MSB-first field layout.

The initialized `rom<T,N>` image format uses this same layout for each word. Images contain one exact-width binary word per line with address zero first; they do not inherit host byte order. Consequently simulator values and Direct-SV `$readmemb` agree on nested aggregate and fixed-point ROM contents. See Initialized synchronous ROMs.

Scalar numeric/raw types and recursively bit-packable structs, vectors, and structural tuples are accepted. Nominal enums, including an aggregate containing an enum, are intentionally rejected by `bitcast`, `pack`, and `unpack`. Enum ordinal storage remains available through ordinary typed values and `reshape`, but raw construction could create an invalid ordinal; no validity policy for that conversion is defined.

At explicitly typed assignment/storage boundaries, ZLang may insert an equal-width raw bitcast only when either source or target is `bits<N>` or flat `vec<N,bit>`. This makes raw wiring concise while preserving the rule that direct signed-to-unsigned, integer-to-fixed, resizing, and rescaling conversions are never implicit. Raw conversion

does not participate in function arguments, operator overloads, generic inference, switch branch unification, or protocol compatibility.

Scalar `reduce(&, value)`, `reduce(|, value)`, and `reduce(^, value)` reduce the stored bits of `bit`, `bits<N>`, `uN`, or `sN` and return `bit`. `parity(value)` is the concise XOR form and also accepts `vec<N,bit>`. Existing vector reduction still reduces elements and returns the element type; fixed-point and struct scalars are not representation-bit reduction operands.

4.9 Current boundaries

Runtime packed-bit selection, runtime/reversed/out-of-range slicing, zero or unresolved packed widths, one-operand or heterogeneous-vector concatenation, runtime reshape, unequal-width bitcast, enum bitcast, and external top enum inputs remain deliberately rejected. These boundaries are diagnosed rather than inferred by an output format. See Known limitations.

5 Expressions, functions, and generics

ZLang expressions are pure typed hardware values unless they explicitly contain a timing construct. Assignment requires an exact canonical target type.

5.1 Operators and selection

From highest to lowest precedence:

1. calls, parentheses, aggregate selection, conversions, functional forms;
2. unary `-` and logical `!`;
3. `*`, and compile-time `/`;
4. `+`, binary `-`;
5. `<<`, `>>`;
6. `&`, then `^`, then `|`;
7. `==`, `!=`, `<`, `<=`, `>`, `>=`;
8. `&&`, then `||` in compile-time/equivalence guards;
9. right-associative `condition ? true_value : false_value`.

Unary `-` is available for numeric scalar/fixed types and may be overloaded for a nominal struct. It is not a general implicit-cast mechanism. Runtime logical `!` accepts exactly `bit` and returns `bit`; it is equivalent to an exact comparison with zero. `/`, `&&`, and `||` are available only where the compile-time/equivalence sublanguage accepts them; they do not infer runtime divider or general truthiness for multi-bit hardware values.

Two-way runtime selection can use `?:` or `mux`:

```
y = select ? a : b
y = mux(select, a, b)
```

The condition is `bit` and both arms have one exact type. Numeric `switch` requires unique fitting keys and an `else` arm:

```
y = switch opcode {
    0 => a
    1 => b
    else => 0
}
```

A switch over a nominal enum uses qualified members and must cover every member. It needs no catch-all arm, which keeps a newly added state from silently taking an old default path:

```
next = switch state {
    State.Idle => State.Header
    State.Header => State.Payload
    State.Payload => State.Done
    State.Done => State.Idle
}
```

Numeric keys in an enum switch, members from another enum, duplicate members, and incomplete coverage are errors. Numeric switches retain their existing mandatory `else` behavior.

Nominal tagged unions use the parallel exhaustive `match` expression. Each arm names its union and variant and binds the exact source-declared payload names; there is no default arm or hidden priority:

```
y = match message {
    Message.Idle => 0
    Message.Data { value } => value
}
```

Typing lowers this surface to an ordinary exact-width `switch` over the union tag plus typed field projections before canonical optimization and backend emission.

Exact matching structs, vectors, and structural tuples also support `==` and `!=`. These comparisons recursively combine the ordinary exact leaf comparisons and return `bit`; mismatched nominal structs, vector lengths, tuple component order/arity, or element types are errors.

5.2 Aggregate value ergonomics

Vector literals and replication use their explicit destination type:

```
pair        : vec<2,u8> = [left, right]
zero_values : vec<4,u8> = repeat(0)
```

`repeat<N>(value)` may state the compile-time length explicitly. The contextual and explicit forms lower to the same existing vector-generation semantics. Struct update is likewise a pure expression over an existing nominal value:

```
next = current with { valid = 1 payload = replacement }
```

It preserves omitted fields and lowers to a complete typed struct construction; it does not create mutable records or a backend-specific update operation. Exhaustive `Struct { field, ... } = value` destructuring similarly lowers to ordinary immutable field bindings. It requires every field exactly once and does not provide partial patterns, renaming, or runtime pattern matching. Flat tuple destructuring `(first, second) = pair` likewise evaluates one RHS and introduces fresh immutable projections. Nested, wildcard, and rest patterns remain outside this bounded surface.

5.3 Pure functions

Functions are top-level, pure, and combinational. The smallest body is one result expression:

```
fn tap(sample : u8, coefficient : u8) -> u16 {
    sample * coefficient
}
```

They may call other non-recursive pure functions. They cannot capture module ports or contain state, clocks, delays, protocols, or pipelines. Return types may be explicit or inferred for both ordinary and generic functions:

```
fn make_tag(high : bits<2>) {
    concat(high, zeros<2>, ones<2>)
}
```

An inferred body is checked without a caller-provided expected type, and its exact final-expression type becomes the function signature. Forward references are resolved independent of declaration order. Direct or indirect inferred return cycles are rejected; ZLang does not attempt a recursive type fixed point. For ordinary functions, explicit and inferred spellings with the same exact signature lower to the same callable identity. Generic specialization identity continues to include the generic declaration form and its exact specialization arguments.

A body may introduce sequential immutable bindings without `let`, `const`, or `return`; the final expression is the result:

```
struct Pair<type T> { left : T right : T }
```

```
fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}
```

Each binding has its exact inferred type, may reference parameters and earlier bindings, and cannot shadow or be reassigned. Forward references and module captures are rejected. This syntax is normalized into the same concrete typed expression/call IR as the nested expression spelling.

Source units are declaration-order independent: functions, structs, enums, and modules may be declared after their users, and a shipped library unit may contain declarations without a hardware top. Parameterized modules can state compile-time obligations at specialization:

```
module Queue<type T,D=4>
where D >= 2 && is_power_of_two(D) {
    // ...
}
```

The constraint is checked against exact type/value arguments and creates no runtime hardware.

5.3.1 Typed constants and statically selected functions

Module and generic-function specializations may carry required named compile-time constants and pure function references:

```
fn widen(x : u8) -> u9 { extend<9>(x) }

fn transform_with<
    type A,
    type B,
    operation : fn(A) -> B
>(x : A) {
    operation(x)
}

module ImageBank<type T,N,IW,image : vec<N,T>> {
    clock clk
    reset rst
    in address : uint<IW>
    out data : T

    rom table : rom<T,N> { read_latency 1 init image }
    table.read_address = address
    data = table.read_data
}
```

These argument kinds are always named: `image=image` and `operation=fn widen`. A generic target may be selected explicitly, for example `operation=fn widen_as<A=u8,B=u9>`. Constants must have the exact declared, recursively bit-packable non-enum type and must evaluate completely during elaboration. A function reference must be pure and match the exact parameter and return types; it is not a runtime function pointer and cannot be stored, returned, compared, or carried through hardware ports.

Type and integer shape parameters resolve before constant/function parameters. The compiler retains a typed binding record containing the canonical constant value or exact callable identity, dependency closure, evaluator schema, and content digest. Source spelling is absent from semantic, artifact, and cache identity. Defaults for constant/function parameters are unsupported.

5.4 Compile-time functions and selection

Compile-time evaluation is bounded and deterministic. Structural intrinsics are built in rather than stdlib overloads:

- `length(vector);`

- `floor_log2(n)`, `ceil_log2(n)`, `index_width(n)`, `is_power_of_two(n)`;
- `pi()`, `sin(x)`, `cos(x)`, `log2(x)`, `log(base,x)`, `exp(x)`, and `sqrt(x)`.

Real intrinsics use a versioned decimal evaluator rather than host binary floating point. A non-integral real enters hardware only through explicit fixed-point quantization. `1 / sqrt(x)` is the compile-time reciprocal square root form used when generating seed or approximation tables.

`index_width(n)` is the storage width for an index into `n` elements: `max(1, ceil_log2(n))`. Its argument must be a positive compile-time integer expression. Integer parameter defaults are resolved in declaration order and may reference only earlier parameters:

```
module Table<N=8,IW=index_width(N)> {
    // Ports and implementation omitted.
}
```

A forward reference such as `module Bad<IW=index_width(N),N=8>` is rejected.

Specialization-time `if` accepts value parameters and canonical type parameters:

```
fn type_flag<type T>(x : T) {
    if T == u8 { 1 } else { 0 }
}

module Stage<D=1> {
    out y : u8
    if D == 1 { y = 1 } else { y = 2 }
}
```

Only the selected branch is semantically checked. Runtime signals are rejected in compile-time conditions with a suggestion to use `?:`, `mux`, `switch`, or a guarded action.

5.5 Functional datapath

Ranges are half-open and compile-time bounded:

```
products = generate(i in 0..8) a[i] * b[i]
doubled  = map(i in 0..8) { a[i] + a[i] }
y        = reduce(+, products)
z        = sum(i in 0..8) a[i] * b[i]
d        = dot(a, b)
```

`generate` and `map` produce `vec<N,T>`. `reduce` supports `+`, `*`, `&`, `|`, and `^`; `sum` is addition reduction. Reduction order is a deterministic source-order balanced tree, so finite-width intermediate types are stable across backends. `dot` uses full-width products followed by that same reduction.

For a nominal struct element, only `sum`/`reduce(+, ...)` is defined. Every internal node resolves the ordinary exact `operator +` overload for the two types produced by its children; the overload result is the exact type presented to the next node. Small and otherwise ineligible reductions retain that complete typed tree directly. Eligible large pure bounded regions instead retain a `FunctionalRegion` plus an `ExactReductionPlan`; the plan records the same balanced topology, exact intermediate result types, and resolved overload/callee identity without cloning every call body. These are two storage forms for one language contract, not two reduction semantics.

There is no component-wise field rule, implicit conversion, identity insertion, or special case for `Complex`. The frozen reduction tree is not reassociated, and explicit quantization remains exactly where the source or overload body placed it.

Range bounds may use resolved module value parameters. Runtime ranges, empty value-producing ranges, and unbounded generation are rejected.

Compile-time iterator arithmetic is mathematical integer arithmetic. It does not require hardware `extend`/`truncate` noise:

```
reversed = generate(i in 0..48) bits[47 - i]
```

For a vector, `values[first..past_last]` is a half-open compile-time range and lowers to the same ordered static element references as the corresponding vector literal. It is not a runtime slice and does not change packed bit-slice syntax `raw[MSB:LSB]`.

A functional iterator is a compile-time integer and may feed a generic value specialization directly or through a bounded constant expression:

```
fn coefficient<K>() -> u8 { K }

values = generate(k in 0..4) coefficient<K=k>()
next   = map(k in 0..4) { coefficient<K=k+1>() }
total  = sum(k in 0..4) coefficient<K=k>()
```

The evaluated value, not the binder spelling, participates in specialization identity. Runtime values are not specialization constants, and an iterator binder cannot shadow a module parameter.

5.5.1 Scalable bounded functional elaboration

Large compile-time `generate`/`map` regions with a binder-invariant result type may use the compact functional representation described above. The compiler still type-checks the region with the ordinary exact rules and counts its logical elements against the same bounded generation budget (currently at most 4096); it does not defer type checking to a backend or create runtime loops. Callable specializations remain ordinary monomorphic definitions, and each use retains its own source origin.

The accepted whole-vector IFFT64 reference exercises an outer 64-lane `generate`, 64 exact products per lane, nominal complex reduction, and one final quantization. N=64 is validated through semantic analysis, canonical round-trip, and the simulator; N=8/N=16 exercise Direct-SV and Verilator. This evidence does not claim full 4096-multiplier N=64 RTL or a production streaming architecture. See the 802.11a validation report for the measured boundary and the later streaming implementation.

A fully compile-time generated vector may initialize an immutable ROM without external source generation:

```
fn coefficients<N>() {
    generate(i in 0..N) truncate<8>(i * i)
}

rom table : rom<u8,N> {
    read_latency 1
    init coefficients<N=N>()
}
```

The complete transitive function/intrinsic dependency identity participates in the ROM and companion hashes. Compile-time functions still have no filesystem, network, process, clock, state, random, or environment access; only the compiler publishes a validated image. The initializer must resolve to exactly the declared vector shape and element type.

5.6 Typed bit and collection composition

Bit slicing and composition are expressions rather than backend spellings:

```
high   : bits<4> = word[7:4]
flag   : bit = word[0]
joined : bits<8> = concat(high, word[3:0])
same   : u8 = bitcast<u8>(joined)
flat   : vec<8,u8> = reshape(matrix)
```

Slices use inclusive compile-time `[MSB:LSB]` bounds. A single packed-bit selection `raw[index]` returns `bit`, uses conventional LSB-zero numbering, and allows a range-proven runtime unsigned index. A fixed-width packed slice `raw[offset +: WIDTH]` returns `bits<WIDTH>` after the same bounds proof. Vector sequence indexing uses the same low-first convention: `vec` element zero occupies the least-significant packed element region. Scalar `concat` places its first operand at the MSB; homogeneous-vector `concat` preserves collection order and returns a vector. `reshape` changes only nested vector shape. `bitcast<T>` uses the canonical layout described in Types and numerics: struct declaration field zero is at the MSB, while vector element zero and tuple `item0` are at the LSB.

It requires exact packed width and performs no resize or scale change. A source or target containing a nominal enum is rejected; enum representation is not a public bitcast contract. `pack` and `unpack<T>` remain low-level compatibility spellings.

For representation-bit checks, `reduce(^, scalar)` and `parity(scalar)` operate on the stored bits of `bit`, raw bits, and signed/unsigned integers. Reducing a general vector retains the existing element-wise reduction semantics.

5.7 Generic structs, functions, and operators

Generic parameters have explicit kinds:

```
struct Pair<type T> { left:T right:T }

fn duplicate<type T>(x:T) { x + x }

module Use {
    in x : u8
    out y : u9
    y = duplicate(x)
}
```

Calls support exact call-site inference or explicit specialization. Generic struct constructors infer their specialization from fields where unambiguous. Every accepted call is monomorphized; there is no runtime dispatch.

A direct integer literal may use a concrete formal argument type after explicit generic arguments or non-literal arguments have fixed that type:

```
first  = same<T=u16>(1)
second = combine(1, existing_u16)
third  = combine(existing_u16, 1)
```

If a type is inferable only from literals, each literal first receives its own minimum-width exact type and normal exact unification applies. Argument order does not select a wider type. Compound expressions never receive this literal-only contextual treatment.

Libraries may overload binary `+`, `-`, `*` and unary `-` for nominal structs. Built-in scalar/fixed operations cannot be replaced. Resolution uses exact type unification, prefers an exact concrete overload over a generic one, performs no implicit conversion, and enforces nominal-owner coherence. Specialized bodies are retained once as deterministic monomorphic callable definitions. Each use site is a typed `Call` carrying the exact callee identity, signature, result type, and its own source origin. Analyses that require concrete arithmetic use the shared bounded call-expansion service; nominal reductions remain opaque to exact reduction rules. Direct-SV emits one `function automatic` per specialization rather than cloning the body at every call site.

`std.math.complex` is ordinary ZLang source defining `Complex<T>`, arithmetic operators, `Butterfly<S,D>`, and explicit component quantization. For example:

```
Complex<fixed<18,16>> * Complex<fixed<16,14>>
    -> Complex<fixed<35,30>>
```

The scale-30 result cannot be added to a scale-16 value until explicitly quantized. Its `complex_sum(values)` helper is a discoverability alias that returns the same typed `Reduce` as `sum(values)`; it adds no compiler behavior. Generics and nominal reductions do not imply rescaling, rounding, saturation, or casts.

Traits, runtime polymorphism, generic methods, function-name overloading, higher-order functions, recursion extensions, and implicit conversions are unsupported.

6 Sequential state, rules, pipelines, and storage

ZLang keeps combinational values, next-state actions, timing, storage, and physical clock ownership explicit in source. Synchronous active-high reset is the default. Each independently declared clock has one reset contract and may own ordinary state in the same module.

6.1 Clock and reset

Single-domain modules declare one pair:

```
clock clk
reset rst
```

Because newlines are ordinary whitespace, the compact adjacent spelling is equivalent and introduces no combined declaration or inferred domain:

```
clock clk reset rst
```

Clock and reset are declared together. Multiple domains require explicit reset, port, and state ownership wherever inference would be ambiguous:

```
clock source_clock
reset source_reset @source_clock

clock destination_clock
reset destination_reset @destination_clock

reg source_count : u32 @source_clock = 0
reg destination_count : u32 @destination_clock = 0

rule TickSource @source_clock when 1 {
    source_count <- truncate<32>(source_count + 1)
}

rule TickDestination @destination_clock when 1 {
    destination_count <- truncate<32>(destination_count + 2)
}
```

An unannotated stateful declaration remains concise in a single-clock module. In a multi-clock module its domain may be inferred from one unique destination or dynamic operand domain; otherwise analysis reports `ZL-DOMAIN-AMBIGUOUS`. Constants are domain-neutral. Combining dynamic values from different domains, including through an unannotated local, reports `ZL-DOMAIN-CROSSING`.

Rules and priorities are scheduled within a domain. Rules in unrelated domains do not acquire cross-clock mutual exclusion or atomicity. FSM-generated state and rules inherit the FSM annotation. Exact `pipeline(N) @clk { ... }`, FIFO, memory, ROM, and CSR state similarly retain one resolved owner; inserted pipeline and balancing registers never act as CDC.

Implicit clock-domain crossing is always rejected. See Hierarchy and protocols.

The recommended asynchronous spelling asserts immediately and releases only after two active clock edges:

```
clock clk
async reset arst_n @clk { polarity active_low }
```

Normal state transition resumes on the third edge after external deassertion. Use the externally synchronized release contract only when the external source already guarantees synchronous deassertion to this exact clock. It removes the internal 2FF while retaining asynchronous reset for state with a declared reset value. The low-level `reset ... { mode asynchronous ... }` spelling retains raw native deassertion for compatibility. Exact polarity, falling-edge behavior, hierarchical propagation, and unsupported formal/target routes are documented in the physical clock/reset contract.

6.2 Registers and next state

```
reg count : u8 = 0
count <- truncate<8>(count + 1)
```

The explicit multi-clock spelling places the annotation before the initializer:

```
reg count : u8 @datapath_clk = 0
```

Expressions read committed beginning-of-cycle state. All accepted `<-` updates become visible together at the active edge. A register without an accepted update holds. Reset restores the declared constant. Multiple unordered writers are rejected.

The reset value is optional. `reg retained : u8` has no generated reset branch or reset sensitivity; its hardware value is unspecified until written. Native two-state simulation starts it from a deterministic zero seed for tool parity, which is not a power-up or reset guarantee in generated RTL.

Use `zlang sim --logic-state` to opt into the per-bit simulation lattice `0`/`1`/`U`/`X`. State without an initializer begins `U` (never refreshed). An accepted state write converts unresolved `U` bits to `X` (refreshed but unknown), while a non-write holds `U`. Exact controlling values still resolve: `U | 1` and `X | 1` are `1`, while `U & 0` and `X & 0` are `0`. `--strict-uninitialized` implies logic-state execution and turns a typed read of either `U` or `X` into an error that identifies which state was observed.

Python callers may use `set_logic`, `get_logic`, `outputs_logic`, and `Program.create(initial_registers=...)`. Packed spellings are MSB-first, case-insensitive `0`, `1`, `u`, or `x`; a single character broadcasts to the full width. Register names use the design's state paths, including explicit child paths such as `child.counter`; ambiguous or non-register names fail before any state is changed. An edge that fails because of an unknown stateful control or native execution error rolls back logic state, event flags, and trace state atomically. CLI equivalents are `--set-logic` and `--initial-reg`. Standard FST/VCD represent both `U` and `X` as `x`; ZLang emits a same-width hidden companion `__zlang_meta/*_u_mask` whose set bits distinguish `U`. `--trace-signal` may be repeated to select the traced surface, and the initial sample is emitted at time zero. The waveform is an event-index visualization: each JSONL event occupies two 1 ns trace ticks, reset changes precede that event's selected clock edges, and falling-edge, active-low, and coincident-domain levels follow the compiled domain contract. It does not reconstruct analog time or simulator delta-cycle regions that are absent from the event schedule.

This is not full Verilog simulation: high impedance (`Z`), electrical resolution/strength, and Verilog event-region semantics are not modeled. Ordinary simulation remains deterministic two-state simulation, and generated Direct-SV is unchanged by these simulator options.

An ordinary output is a combinational wire. Output-visible state is declared explicitly with `out reg`:

```
out remaining : s32              // combinational wire
out reg accepted : s32 = 0       // resettable stored output
out reg retained : u8 @data_clk  // stored output with no reset branch
```

A registered output is one state resource and the public output port at the same time. It is readable inside the module, accepts `<-` updates, and holds when no accepted action updates it. Its optional initializer is its reset value; omitting it has the same unspecified-hardware-reset semantics as an ordinary register without an initializer. The type must be register-storable. Ready/valid, credit, packet, request/response and other protocol endpoints are not `out reg` ports; their individual protocol fields retain their existing ownership and flow-control rules.

ZLang `out reg` names actual architectural storage. It is not the historical Verilog/SystemVerilog `output reg` declaration category: a ZLang registered output has hold behavior, one explicit state owner, and the same clock/reset and atomic-update rules as an ordinary ZLang register.

Control registers may use nominal enums. Reset and next-state values must be members of the exact same declaration, and an exhaustive enum `switch` is the preferred finite-state-machine spelling:

```
enum Phase { Idle Active Done }
reg phase : Phase = Phase.Idle
phase <- switch phase {
    Phase.Idle => start ? Phase.Active : Phase.Idle
    Phase.Active => finish ? Phase.Done : Phase.Active
    Phase.Done => Phase.Idle
}
```

The semantic simulator and Direct-SV share the declaration-order ordinal encoding. The source-authored APB bridge in `stdlib/bus/apb.zhl` uses an enum for its internal phase without exposing it through the external bus ABI.

For control-oriented state, `fsm` is concise syntax for the same enum register and guarded atomic rules:

```
enum TxPhase { Idle Header Payload Done }

out reg remaining : u32 = 0

fsm phase = TxPhase.Idle {
    Idle {
        when start -> Header { remaining <- length }
    }
    Header { when header_sent -> Payload {} }
    Payload {
        priority {
            when abort -> Idle {}
            when last -> Done {}
        }
    }
    Done { -> Idle {} }
}
```

The qualified initial member in `fsm phase = TxPhase.Idle` infers the exact nominal enum type. The explicit compatibility spelling `fsm phase : TxPhase = Idle` remains accepted; neither form infers from guards, actions, or transition targets.

Every member appears exactly once. A state body is `hold`, one transition, or an explicit `priority` block; source order never creates hidden priority. The state update and transition actions form one ordinary atomic rule, reset restores the declared initial member, and direct writes to the generated state register are rejected. `fsm` is lowered before canonical/backend IR and does not add a second scheduler or runtime procedural `if`.

The production Wi-Fi `IeeeIFFTInputStrip` is a second source-authored witness: its `Data`/`Flush` FSM atomically starts and retires a 64-transfer zero-frame flush, holds under backpressure, and returns to `Data` after the last accepted sample. Mid-flush reset restores the declared initial state and discards the incomplete protocol epoch.

6.2.1 Runtime-indexed vector register updates

A one-dimensional vector register supports one range-proven runtime element write in an action group:

```
reg samples : vec<64,u8> = generate(i in 0..64) 0

when capture {
    samples[index] <- value
}
```

This is exactly `samples <- VectorUpdate(samples,index,value)` over the beginning-of-cycle snapshot. The unsigned index must be statically proven in range. A second dynamic or static write to the same vector register conflicts; nested paths, slices, runtime-selected instances, and automatic memory inference remain outside this bounded form.

A rule guard may establish the required bound through a simple unsigned comparison and conjunction:

```
when (index < 64) & capture {
    samples[index] <- value
}
```

That fact is scoped to this rule only. The bounded refinement understands these simple comparisons/conjunctions; `index <= 64`, disjunctions, and uses outside the guarded rule do not prove the required `0..63` range.

6.3 Delays and fixed pipelines

```
delayed = delay<2>(value)
piped = pipeline(3) { expression }
```

`delay<N>` adds exactly `N` zero-reset scalar stages. `pipeline(N)` is an exact visible-latency contract: for a supported pure scalar expression DAG, planning may place real computation boundaries within those `N` cycles and inserts

the required reconvergent-path balancing. It never changes the requested latency or II, and `pipeline(1)` remains exactly one cycle. Unsupported nodes fail closed; the emitter does not fall back to an unreported whole-expression output delay. Outside a scheduled pipeline region, non-timeless operands at an operator must still be latency-aligned explicitly.

Compiler-selected latency/resource alternatives use `implement { expression intent { ... } }`. Scalar `pipeline(auto)` is retired; protocol `transform pipeline(auto, ...)` is a separate ready/valid construct.

6.4 Exact module timing contracts

A scalar datapath module may publish one exact contract shared by all of its wire outputs:

```
module TimedChild {
    clock clk
    reset rst
    in x : u8
    out y : u8

    timing {
        latency 4
        ii 1
    }

    y = pipeline(4) { x }
}
```

The contract describes observable behavior; it does not itself insert registers or authorize retiming. Constants are timeless, ordinary input-dependent logic is known at latency zero, and explicit `delay<N>`/`pipeline(N)` adds exactly `N`. A supported pipeline region may use its selected schedule to place and balance those internal cuts. Timeless values may join a known path; joins outside that graph still require equal known latency.

Registers, rules, FIFO/memory/ROM observations, protocols, and uncontracted child outputs are `unknown` under the current bounded contract. A positive latency requires exactly one clock/reset domain. Only `ii 1` is accepted. A contracted child adds its declared latency to the common known latency of its bound scalar inputs; an unaligned parent join is rejected instead of silently balanced.

This is distinct from `implement` constraints, target estimates, and measured evidence. A selected profile must preserve an exact module contract as immutable public behavior.

6.5 Guarded atomic actions

The concise action form is:

```
when increment {
    count <- truncate<8>(count + 1)
}
```

`<-` is exclusively a state update. It may target an ordinary register, an `out reg`, or a supported indexed vector-register element. An ordinary `out` wire is never stored and therefore cannot be updated with `<-`. A transient rule/FSM output uses `drive` instead:

```
out fired : bit

rule publish when accept {
    drive fired = 1
}
```

The driven wire is zero when no owning action fires. It participates in the same atomic scheduling, conflicts and explicit priority as the rule's other effects. `drive` cannot target a register or `out reg`, and a registered output cannot also have a combinational assignment. Because changing a wire update to stored state is a semantic choice, the compiler reports both alternatives but does not apply an automatic editor fix.

`drive` is a logical transient output assignment. It does not model a tri-state driver, high impedance (`Z`), electrical

or pad drive strength, analog behavior, or Verilog multiple-driver resolution. Its zero default is therefore pulse/event semantics, not retained state.

Form	Cycle semantics	What it is not
`out value : T with value = expression`	Continuously defined combinational value	Stored state
`out reg value : T [@clock] [= reset] with value <- expression`	Clocked architectural state that holds when not updated	Verilog's historical output declaration category
`drive value = expression` in an action	Scheduled transient wire effect; zero when no owning action fires	Tri-state/electrical drive or implicit storage

Named compatibility spelling remains supported:

```
rule increment_count when increment {
    count <- truncate<8>(count + 1)
}
```

Action blocks may select effects recursively with runtime `when`, `else when`, and `else`:

```
fault_update: when fault {
    when valid_state {
        overflow_state <- 1
    } else {
        valid_state <- 1
        overflow_state <- 0
        sticky_state <- fault_code
    }
} else when clear_valid {
    valid_state <- 0
    overflow_state <- 0
} else when clear_overflow {
    overflow_state <- 0
}
```

An `else` binds to the nearest unmatched `when`, and an `else when` chain selects its first true predicate. A false `when` without an `else` contributes no effects; later statements in the enclosing action block are still considered, so multiple sibling conditionals can contribute to the same transaction.

Read this as two operations that never feed back into each other: predicates first choose the effects, then the scheduler either commits the complete chosen transaction or commits nothing. Storage readiness and rule conflicts belong only to the second operation; they can suppress a transaction but can never make an `else` branch run.

Every guard and operand reads the same pre-edge snapshot. The complete tree retains one outer `Rule`, one rule-fire identity, and one `ActionGroup`; selected effects and unconditional surrounding effects commit together or not at all. If no effect is active, the rule does not fire. Source order creates neither state visibility nor implicit priority, and reset suppresses the complete group.

Branch choice depends only on the predicates. If the selected branch contains an illegal FIFO or memory action, the whole action group is suppressed; it does not fall back to an `else` branch because another branch's storage action would be ready. The scheduler checks conflicts and storage legality using only active effects. Explicit `drive` output writes are scheduling resources too: a rule that loses a priority conflict on an output cannot commit its otherwise unrelated state effects.

Each runtime action condition must have exact type `bit`. Opposite structured branches may write the same resource, but writes on potentially overlapping paths are rejected; the compiler does not attempt arbitrary Boolean theorem proving. An explicitly empty branch is a valid no-op, while an entire action tree with no possible effect is rejected.

Runtime action selection is distinct from both other selection forms:

- compile-time `if` chooses declarations or values during elaboration and is not an action statement;

- `condition ? a : b`, `mux`, and `switch` select pure combinational values;
- runtime `when` selects effects that participate in one cycle-level atomic transaction.

Rules read beginning-of-cycle state. If the guard is true, all resource actions are legal, and scheduling permits the rule, its actions commit atomically. Reset suppresses all rules. Conflicting writers require explicit priority:

```
priority {
    clear: when clear_request { count <- 0 }
    increment: when increment_request {
        count <- truncate<8>(count + 1)
    }
}
```

Priority suppresses a lower enabled rule only when it conflicts with a fired higher rule. It is not procedural `if/else`: nonconflicting enabled rules may fire together.

A simple ordered chain may instead be written as:

```
priority first > second > third
```

This syntax expands to the existing adjacent edges `first > second` and `second > third`. It does not invent extra edges or infer priority from source order; retain the block/pair form for any other priority graph.

6.6 FIFOs

```
fifo queue : fifo<u8,4>
queue.data = rx.payload
queue.push = rx.transfer
queue.pop = tx.transfer
```

Read-only observations include `.front`, `.count`, `.full`, `.empty`, `.ready`, `.valid`, `.overflow`, and `.underflow`. A legal simultaneous pop/push while full preserves occupancy and ordering. An empty pop cannot consume a same-cycle push.

FIFO depth may be a positive compile-time value expression. Backends receive the resolved integer, not a hardware parameter.

Rules can instead own FIFO operations atomically:

```
rule replace when rotate {
    queue.pop()
    queue.push(queue.front)
    accepted <- 1
}
```

Do not mix rule-owned actions with globally driven `.push`/`.pop` controls. An illegal resource action suppresses the whole rule.

Rule scheduling selects the lexicographically highest-priority **legal atomic set**, not each rule independently. Thus a push on a full FIFO may fire with a compatible lower-priority pop on the same edge; a push alone remains blocked. The compiler constructs exact selection regions for empty, interior, and full occupancy without enumerating every count or rule subset. If exact scheduling exceeds its bounded work budget, compilation reports `ZL-STATE-SCHEDULER-LIMIT` rather than emitting an approximate circuit.

6.7 Writable memories

One-read/one-write memories may optionally expose a byte write mask. The existing one-cycle, reset-cleared form remains the default:

```
in write_mask : bits<4>
memory table : mem<u32,256> {
    read_latency 1
    collision write_first
}
```

```
table.read_address = read_address
table.write_enable = write_enable
table.write_address = write_address
table.write_data = write_data
table.write_mask = write_mask
```

The mask type is exactly `bits<ceil(element_width / 8)>`. `write_mask[0]` controls the least-significant byte. For a width that is not a multiple of eight, the final mask bit controls only the remaining live most-significant bits; padding bits are never stored. Disabled lanes retain their old bits. For a same-address `write_first` access, the registered read result is the post-mask merged word, not the unmasked input. Omitting `write_mask` preserves the existing full-word write behavior. Consequently, unmasked and masked memory elements may have any positive recursively bit-packable, non-enum width.

Rule-owned memories accept the same operation as an optional third operand:

```
rule store when enable { table.write(address, data, byte_mask) }
```

Within a masked rule-owned memory, the compatible two-operand form `table.write(address, data)` means an all-lanes write. The mask does not add a read enable, another port, a new clock domain, or target-specific BRAM mapping.

`read_latency` counts active edges in the memory's read domain between a captured address and the visible result. These are compiler/simulator/Direct-SV contracts, not promises that a particular FPGA RAM primitive will be inferred:

Storage form	Accepted `read_latency`	Tested behavior and physical boundary
Globally controlled `mem`, including named same-clock ports	`0..16`	`0` is combinational; `1` is registered; `2..16` add exact read-domain register stages. Cycle tests cover `0`, `1`, `2`, and the upper bound `16`; Direct-SV is linted.
Rule-owned legacy `mem`	Exactly `1`	A selected read action captures at an edge and holds its result otherwise; rule/storage simulations and Direct-SV tests cover it.
`async_mem` 1W1R	`1..16`	Reader-domain registered output; cycle tests cover `1`, `2`, `3`, and `16`, including unrelated clocks. Generic Direct-SV supports `old`; stronger cross-clock collision claims need exact target evidence.
Immutable `rom`	Exactly `1`	Synchronous table read; compile, simulation, and Direct-SV tests cover it.
Internal memory of `async_fifo`	Fixed `1`	The FIFO prefetch controller accounts for the registered read; this is not a user-settable FIFO latency.

The current explicit Xilinx 7-Series synchronous-memory physical route accepts only `read_latency 1` and matching reset, mask, port, and collision capability. Other legal latencies can use generic RTL under a preferred policy; a required native route rejects them. `read_latency 0` never claims synchronous BRAM.

6.7.1 Named same-clock ports

Leaving the body without port declarations preserves the legacy controls and their generated behavior. A memory may instead declare up to eight named logical ports:

```
memory table : mem<u32,1024> @clk {
    read_write_port a
    read_write_port b
    init RESET_WORD
    read_latency 1
    collision old
```

```
    write_priority a > b
}

table.a.read_enable = a_read
table.a.write_enable = a_write
table.a.address = a_address
table.a.write_data = a_data
a_data_out = table.a.read_data
```

`read_port`, `write_port`, and `read_write_port` state the exact access shape. All named ports of an ordinary `mem` have one resolved clock domain. When more than one port can write, `write_priority` must list every writer exactly once. Priority affects only writes to the same address; simultaneous writes to different addresses both commit. `collision old`, `new`, and `no_change` select the same-clock registered-read result. The older spellings `read_first` and `write_first` remain accepted aliases for `old` and `new`.

Named-port masks retain the arbitrary-bitwidth lane rule above. Mixed port widths and asymmetric depths are rejected. Rule-local memory actions remain a property of the legacy implicit 1R1W form and cannot be mixed with named ports.

An optional `init VALUE` supplies one exact compile-time element value for every cell. It may be a typed module value parameter. With `contents clear`, generic/register-array hardware writes that value to every cell on reset. With `contents preserve`, it is only the deterministic power-up value; FPGA block RAM maps it to bitstream initialization and does not pretend that a runtime reset rewrites the array. Omitting `init` retains the exact zero value used by the existing reset/initialization policies. Full per-address images remain the role of immutable `rom` in the current language contract.

When a memory exactly matches a target resource, that resource is used. A `1W+nR` shape otherwise becomes coherent replicated 1R1W storage; other same-clock multiwrite shapes may use a deterministic register array, read muxes, and priority gates when the logical storage is at most 4096 bits. Larger unsupported shapes fail with a diagnostic recommending explicit banking, arbitration, or standard-library composition. No automatic banking is performed because arbitrary same-bank collisions need a protocol contract.

6.7.2 Explicit dual-clock memory

`async_mem<T,N>` is a separate, deliberately narrow semantic resource:

```
memory table : async_mem<u32,1024> {
    write_port wr @write_clk
    read_port rd @read_clk
    init INITIAL_WORD
    read_latency 1
    collision old
    reset { contents preserve read_data clear }
}

table.wr.enable = write_enable
table.wr.address = write_address
table.wr.data = write_data
table.wr.mask = write_mask
table.rd.address = read_address
read_data = table.rd.data
```

It has exactly one write-only port and one read-only port in different explicit domains. Cells change only on the writer edge; `rd.data` changes only on the reader edge and has destination-domain provenance. The contents reset policy belongs to the write domain and the read-result policy to the read domain. Direct use of foreign-domain controls is rejected; a pipeline register is not a CDC primitive.

`async_mem` requires `read_latency` in `1..16`. The generic implementation captures the cell on the reader edge and, for latency greater than one, shifts the captured value through reader-domain output registers. This adds visible latency without changing the contents or crossing domains. `read_latency 0` is invalid for `async_mem`; it would be an unregistered cross-clock read, not a block-RAM read port.

For coincident logical edges the simulator uses a pre-edge snapshot: `old` returns the prior cell, while `new` forwards

the coincident write. This is a precise digital model, not an analog metastability or silicon timing guarantee. Portable generic Direct-SV publishes `old`; stricter or other collision claims require an exact target capability. Asynchronous 2RW, mixed-width ports, ECC, and dual-clock ROM remain deferred.

```
memory table : mem<u8,16> {
    read_latency 1
    collision read_first
}

table.read_address = read_address
table.write_enable = write_enable
table.write_address = write_address
table.write_data = write_data
read_data = table.read_data
```

Alternatively, one memory can be owned by atomic rules:

```
rule fetch when read_enable {
    table.read(address)
    accepted <- 1
}

rule store when write_enable {
    table.write(address, write_data)
}
```

`read` registers `table.read_data` one cycle later and it holds when no read fires. `write` commits at the selected edge. One selected read and one selected write may fire together; same-address behavior follows the declared collision mode. Their operands and all other rule effects observe the same pre-edge snapshot. Reset suppresses actions. With the default profile it clears both cells and `read_data`.

Global controls and rule actions cannot be mixed. The bounded scheduled form supports one memory per module, alongside registers and scheduled FIFOs; two reads or two writes conflict and require existing explicit rule priority.

Executable memory elements may be recursively bit-packable non-enum aggregates, as well as scalars; depth is a power of two and at least two. A globally controlled ordinary `mem` accepts `read_latency` in `0..16`: zero is a combinational read; each positive value is an exact number of domain-local read edges from address capture to the visible result. Rule-owned memories remain exactly one-cycle because their read action is selected at an edge. Zero-cycle ordinary `mem` is a generic combinational storage behavior, not a native synchronous BRAM claim. Native target binding requires an explicitly matching clocked-read capability. Until a target mapping proves its internal output-register configuration and any additional fabric stages, a requested latency greater than the advertised native latency uses generic storage under a preferred policy or fails under a required policy.

Cell and visible read-result reset behavior may be selected independently:

```
memory table : mem<u32,256> {
    read_latency 0
    collision read_first
    reset {
        contents preserve
        read_data preserve
    }
}
```

If the `reset` block is present, both directives are required. If it is absent, the exact default is `contents clear` and `read_data clear`. Reset always suppresses writes and scheduled actions. At latency zero, `read_data clear` masks the combinational result while reset is asserted; `read_data preserve` leaves the addressed preserved cell visible. At latency one, the corresponding policy clears or holds the result register. `read_first` observes the old cell before a coincident write edge; `write_first` observes the fully byte-mask-merged write word.

The simulator and generic Direct-SV initialize executable preserved memory state to its declared uniform `init` word, or zero when omitted, but runtime reset does not recreate that initialization under `contents preserve`. The

Xilinx same-clock true-dual mapping therefore requires preserved contents; Vivado maps its source initializer to BRAM initialization. Target selection stays fail-closed when a resource does not advertise initialization support. Full/partial writable-memory images, automatic banking, unbounded multiport memory, and asynchronous 2RW are not part of this hardware surface. Bounded named-port selection is described above. The writable-memory contract defines the hardware boundary.

6.7.3 Replicated read ports, wrappers, and banking

Named ports now provide a bounded native logical model; source composition is still preferable when banking or arbitration policy is architecturally significant. The validated `ZtpuBankedMemory` uses four banks and two read replicas per bank: both replicas receive the same decoded, byte-masked synchronous write, while each read address selects its own replica and bank. This produces eight distinct physical 1R1W memories, one shared leaf specialization, two combinational read results, and one logical write port.

The concrete witness contains 256 32-bit words, split into four banks of 64 words. Its runtime reset suppresses writes and preserves both cells and combinational read data; executable simulation begins from deterministic zero contents. The semantic simulator and Direct-SV agree on full and per-byte writes, independent reads, same-address `read_first` behavior, reset, and replica coherence. This is source composition, not a new memory semantic or backend name-based rewrite. The ordinary stdlib modules `StorageDualPortMemory`, `Storage2R1WMemory`, and `StorageAsyncMemory1W1R` expose common shapes while lowering through the same memory IR. `StorageAsyncFifo` similarly wraps the existing ready/valid `async_fifo(D)` crossing; a normal `fifo<T,N>` never becomes asynchronous.

The FIFO crossing now decomposes into an internal typed `async_mem` with independent write/read clocks, `read_latency 1`, and a one-slot prefetch controller. It is not a second public memory declaration in the module. The selected AMD 7-Series `Xilinx7AsyncFifoRAMB18` or `Xilinx7AsyncFifoRAMB36` architecture binds only this internal FIFO memory to a matching independent-clock 1W1R RAM with `DO_REG=0`; unsupported width/depth/configuration fails under `required` policy and remains generic RTL under `preferred`. This route does not imply that arbitrary public `async_mem` has a native cross-clock collision guarantee.

Target-memory closure remains separate. In particular, the existing promoted OpenRAM/target macro contract does not yet match this zero-latency, preserve-on-reset profile, so the witness makes no BRAM/SRAM inference, QoR, or physical-macro claim. A byte-addressed external wrapper and target-specific latency/collision adaptation are not currently supported by this mapping.

6.8 Initialized synchronous ROMs

An immutable lookup table is declared separately from writable memory:

```
rom twiddles : rom<Complex<Twiddle>, N / 2> {
    read_latency 1
    init fft_twiddles<T=Twiddle,N=N>()
}

twiddles.read_address = phase
selected_twiddle = twiddles.read_data
```

The initializer must specialize at compile time to exactly `vec<depth,element_type>`. Elements may be bit-packable scalar/fixed values, vectors, or concrete structs; protocol/state values and nominal enums are rejected. Depth is positive and may be one or non-power-of-two. The address is an unsigned `uint<max(1,ceil_log2(depth))>` value whose proven range remains inside the declared depth.

The read is a **one-cycle synchronous read**. During reset the registered read result is zero. Reset never changes the immutable contents; after reset, the first non-reset result corresponds to the preceding accepted non-reset address. There is no hidden second output register.

Generated SystemVerilog uses one companion image. It contains one exact-width binary word per line, address zero first. Struct declaration field zero occupies the most-significant bits; vector element zero and tuple item zero occupy the least-significant component bits recursively. Fixed-point values retain their raw signed or unsigned bit pattern. Direct-SV `$readmemb` consumes that image. The CLI publishes companions beside the selected output, and artifact metadata retains the initialization dependency, evaluator, content, and file hashes. Missing or colliding companion files fail closed.

Writable or partial initialization, reloadable/asynchronous/multiport ROM, enum elements, and automatic BRAM/storage exploration remain deferred.

6.9 Stateful hierarchy

Single-domain scalar child modules may contain registers and rules. A compile-time indexed instance array may contain bounded scalar sequential or primitive ready/valid children with one matching clock/reset domain; each physical instance has independent state and reset. Aggregate scalar outputs and mixed scalar/ready-valid ports retain their exact typed leaves. Storage-only child arrays may own one FIFO, synchronous memory, or initialized ROM. The scheduled-FIFO profile may also combine FIFO actions with ordinary register writes/rules because both use the same `ResolvedTransition`; legacy globally controlled storage plus user state remains rejected.

One outer array may contain a bounded same-domain scalar or direct-ready-valid hierarchy, including an inner scalar compile-time array. First-level in-order request/response requester/responder arrays are supported with scalar wire peers and explicit indexed connections. Nested storage/CSR/aggregate protocols, nested or out-of-order request/response, other non-RV protocols, CDC, runtime-selected instances, and cross-module atomic scheduling remain fail-closed. See Hierarchy and protocols.

7 Physical clocks and resets

ZLang's `ClockDomain` records the complete physical contract of one clock and its reset. Concise declarations retain the default reset contract:

```
clock clk
reset rst @clk
```

normalizes to rising-edge clocking, synchronous active-high reset, native release, and unspecified power-up state.

7.1 Recommended asynchronous reset

The concise asynchronous form is safe for ordinary state. In a multi-clock module each annotated domain owns an independent conditioner:

```
clock clk
async reset arst @clk
```

It means asynchronous assertion followed by deassertion synchronized through two registers. Reset remains active on the first two active clock edges after the external pin is deasserted; normal state transition resumes on the third edge. Reassertion at any point immediately restarts that release sequence.

The external pin may be active-low:

```
clock clk
async reset arst_n @clk {
    polarity active_low
}
```

If reset deassertion is already synchronized to this exact clock outside the generated module, state that physical contract explicitly:

```
clock clk
async reset arst_n @clk {
    polarity active_low
    release externally_synchronized
}
```

This retains asynchronous assertion for resettable state but emits no internal two-register release conditioner. The annotation is never inferred: every destination clock domain must receive a release synchronized to that clock. Omitting `release` retains the safe internal 2FF default.

A register participates in reset only when it declares a reset value:

```
reg control : u8 = 0 // asynchronous reset to zero
reg datapath : u8    // no reset branch or reset sensitivity
```

The unreset register continues to accept enabled clocked updates while reset is asserted. Its RTL power-up value is unspecified until the design writes it. Native two-state simulation uses a deterministic zero startup seed, which is not a source-level power-up or reset guarantee. The opt-in logic-state mode represents unrefreshed bits as `U`; strict observation rejects `U` or computed `X` rather than collapsing either to a binary value. See Registers and next state.

A falling-edge clock uses falling edges for the two release cycles as well. Polarity affects the external pin and generated RTL; the simulator still accepts a logical `True` for asserted reset.

The lower-level compatibility spelling remains available for a raw native asynchronous assertion/deassertion contract:

```
clock clk { edge falling }
reset rst_n @clk {
    mode asynchronous
    polarity active_low
    power_up unspecified
}
```

This full block does not insert synchronized release. When either low-level physical block is present, every clause in that block is required. Raw active-low protocol helpers gate transfers with the deasserted high level; they do not reinterpret the physical pin as active-high.

7.2 Hierarchy inheritance

A stateful child that deliberately declares no physical clock/reset may inherit the enclosing module's sole exact clock domain. This is source ownership, not an implicit adapter: edge, assertion mode, polarity, release policy/cycles, power-up policy, and physical names become the child's typed contract and specialization identity. Reusable stateful RegBus, AHB-Lite, AXI4/AXI4-Lite, AXI burst, AXI-Stream, APB, and Wishbone cores use this mechanism; source-owned flat-pin adapters and concrete application/witness tops remain explicit physical-domain owners.

Inheritance fails closed when the parent has no unique domain. A child that declares its own domain must still match one exact parent domain; no reset conversion or CDC is inferred. The Python compilation/session API may supply one typed `root_domain` when an inherited-domain core itself is selected as a direct test or tool top. This does not create a source-language default or bypass validation: the supplied contract participates in the exact compilation cache key and produced artifacts, while absent or ambiguous ownership remains an error.

The generated implementation artifact records the contract identity, external RTL clock/reset paths, edge, assertion mode, polarity, release policy/cycles, power-up policy, and source origin. Component and physical-instance records reference that identity.

7.3 Boundaries

One clock domain has exactly one reset. Declaring both `reset` and `async reset` for that domain is an error. Direct-SV supports multiple independently conditioned asynchronous domains; current formal execution still skips an asynchronous goal in a multi-domain module because no cross-domain reset-epoch relation is claimed. Implicit reset crossings, reset combiners, configurable synchronizer depth, glitch filters, power-on reset, and macro-selected RTL semantics are not supported.

`power_up reset` remains typed but fails closed because no common portable synthesizable initialization mechanism is frozen. DSP48 physical reset pins, elastic or variable-latency `pipeline(auto)` equivalence, CDC/reset-refinement proofs, and target BRAM reset pins remain fail closed. So do multi-domain asynchronous formal execution, general hierarchical semantic-reference equivalence, and every route with missing or incompatible domain manifests, bindings, assumptions, or observations. formal-aware selection `available` records unavailable evidence without changing eligibility; required policies fail unless the existing exact semantic-reference equivalence route executes at the requested level. No new formal property, observation, or equivalence family is introduced.

8 Hierarchy, protocols, and explicit CDC

ZLang represents modules, endpoints, ownership, clock domains, connections, and physical instances explicitly before backend lowering. Connections are never inferred by guessing generated RTL names.

8.1 Modules and instances

Modules may have compile-time type and value parameters:

```
module Engine<type T, DEPTH=4> {
    in  value : T
    out copy  : T = value
}
```

Instantiate and bind scalar inputs explicitly or by same-name shorthand:

```
inst engine : Engine<T=u8,DEPTH=4> { value }
result = engine.copy
```

Specialization identity is distinct from physical instance identity. A single-domain parent may supply an unambiguous child clock/reset implicitly; multi-domain hierarchy requires explicit compatible domains. Child protocol endpoints use `connect`, not scalar binding syntax.

A reusable component fingerprint covers its typed hardware and the deterministic transitive closure of callables that hardware actually invokes. Functions that are merely visible through a parent/import context do not participate. Thus the same parameterless child reached through different parents is emitted once, while different value/type specializations or a different reachable helper body remain distinct and fail closed on an identity conflict.

When every intermediate child conforms to one exact named interface with one protocol input and one protocol output, an option-free path can be written as:

```
input -> decode -> execute -> output
```

This lowers to the ordinary pairwise typed edges `input -> decode.rx`, `decode.tx -> execute.rx`, and `execute.tx -> output`. Intermediate names are physical instances, not guessed ports. Arrays, unnamed or ambiguous signatures, edge options, adapters, buffering, and crossings remain explicit.

One-dimensional compile-time instance arrays are structurally unrolled:

```
inst lane[N] : Lane<W>
generate(i in 0..N) {
    lane[i].x = values[i]
}
outputs = generate(i in 0..N) lane[i].y
```

Structural generation may be nested and may contain compile-time `if`/`else`. Both are elaborated by the same bounded unroller; they do not create runtime loops or dynamic topology:

```
generate(i in 0..N) {
    generate(j in 0..N) {
        if i == j { lane[i].x = values[j] }
    }
}
```

The existing total expansion limit applies across the complete nested tree. Exceeding it reports a bounded compile-time-generation diagnostic rather than partially elaborating the design.

A runtime selector may project one exact bit-packable wire output. This is a read-only mux over the already elaborated children, never a dynamic physical instance:

```
selected = lane[select].output
```

Every `lane[i]` still exists, receives its compile-time bindings, and advances its own state independently. The selector neither routes child inputs nor gates a child clock, reset, rule, or storage action. The explicit equivalent is `outputs = generate(i in 0..N) lane[i].output` followed by `selected = outputs[select]`; both spellings retain the same physical child identities.

The bounded supported form covers:

- combinational scalar children, including exact aggregate scalar outputs;
- single-domain scalar children with registers/rules;
- primitive ready/valid children, including scalar wire ports beside the endpoint and the existing one-FIFO profile;
- legacy storage-only children owning one FIFO, synchronous memory, or initialized ROM;
- the scheduled-FIFO profile in which FIFO actions and ordinary register writes share one existing `ResolvedTransition`; and
- an outer one-dimensional array whose element contains a bounded same-domain scalar or direct-ready-valid hierarchy. Inner compile-time scalar arrays are retained as distinct physical paths rather than flattened by generated name.

The first-level request/response profile is also bounded and explicit. Each array child has exactly one `ordering in_order` endpoint plus scalar wire ports, and each connection names compile-time indexed requester and responder elements:

```
generate(i in 0..N) {
    requester[i].issue = issue[i]
    responder[i].accept = accept[i]
    connect requester[i].mem -> responder[i].mem
}
```

Every physical element retains its own outstanding ledger and reset epoch. Out-of-order matching, an additional protocol on the same child, nested or transitive request/response arrays, and unindexed endpoint references fail closed. This profile has an executable capability-registry witness and is covered by compiler, simulator, and backend validation.

Legacy globally controlled storage still cannot be combined with user registers/rules. Credit and other non-RV protocols, CSR or storage below a nested array element, CDC arrays, runtime-selected inputs or protocol endpoints, cross-module atomic scheduling, and incomplete bindings are rejected.

Concise declarations normalize to the same hierarchy IR when unambiguous:

```
frontend : AXI4LiteToRegBus<32,32>
axi : AXI4Lite<32,32>.slave @clk
axi -> frontend.axi
```

Explicit `inst` remains the escape hatch for namespace ambiguity.

8.1.1 Typed external modules

A bounded scalar-wire component may declare portable behavior with an ordinary pure ZLang model:

```
extern module VendorAdd : AddIfc { model add_model }
```

The concise source form is `extern module VendorAdd : AddIfc { model add_model }`.

The applied interface and model produce one exact semantic signature. Physical SystemVerilog is supplied separately through a hash-validated `ExternalPhysicalMapping`; inline HDL never defines ZLang semantics. The supported form has non-parameterized, clockless scalar inputs and one scalar output in Direct-SV. State, protocols, arrays, and generic external components fail closed.

8.2 Clock and reset boundary

Legacy `clock clk` plus synchronous active-high `reset rst` works through the production Direct-SV backend. A single-domain parent may supply that exact domain to a child only when the mapping is unambiguous; this is domain inheritance, not implicit CDC. Reusable stateful standard-bus cores use this same rule; source-authored pin adapters and concrete application tops remain the external domain owners. One non-default falling-edge, asynchronous, or active-low physical contract is typed and emitted by Direct-SV. Concise `async reset` adds one

root-owned two-edge release conditioner and passes the conditioned reset through the closed child ABI; the raw asynchronous compatibility form remains distinct. Instance arrays require one compatible inherited domain unless they are purely combinational. Multiple independently conditioned asynchronous domains are supported in Direct-SV. Every domain of a stateful child must match exactly one parent physical clock/reset contract; no implicit reset crossing is inserted. Multi-domain asynchronous formal execution, power-on reset, and implicit reset crossing remain fail-closed.

8.3 Public top ABI

The selected top always exposes integration-friendly typed leaves. Struct fields and structural tuple components, including those inside protocol payloads, become individually named ports; tuple paths use `item0`, `item1`, and so on. A vector leaf remains one multidimensional packed SystemVerilog array; for example `vec<8,u8>` becomes `logic [7:0][7:0] samples`. It is neither an anonymous 64-bit public bus nor eight separately named ports. Nested `vec<Struct>` values become one array per struct field, and `vec<Tuple>` values become one array per tuple component.

Direct-SV RTL uses this public `TopPhysicalABI`. The conversion follows the canonical packing order with element zero in the least-significant region: for a descending packed vector dimension, source element zero maps to physical index `0`. There is no source annotation, compiler option, or backend-specific preprocessor branch selecting another public ABI.

8.4 Ready/valid

```
in  rx : rv<u8>
out tx : rv<u8>

tx.payload = rx.payload
tx.valid = rx.valid
rx.ready = tx.ready
```

For an input endpoint, payload/valid enter the module and ready leaves it. Output ownership is reversed. `.transfer` is a read-only protocol property meaning `valid & ready`. A source stalled with `valid=1` and `ready=0` must hold payload and valid stable.

Direct composition names source then sink:

```
connect rx -> tx
connect buffered_rx -> buffered_tx { buffer 2 }
```

A ready/valid buffer is real FIFO state; it does not appear implicitly.

8.5 Credit and request/response

`credit<T,N>` provides pulse-return flow control. `.transfer` is the physical send after credit gating, `.credits` is the current bounded count, and `.return` returns one credit. Counters begin at `N`.

Adapters are explicit:

```
connect rx -> tx { adapter rv_to_credit }
connect crx -> rtx { buffer 2 adapter credit_to_rv }
```

Request/response endpoints declare ordering and outstanding capacity:

```
interface mem : request_response<Request,Response> {
    max_outstanding 2
    ordering in_order
}
```

The requester owns request payload/valid and response ready; the responder owns request ready and response payload/valid. Request and response channels expose their own read-only `.transfer` events. Directional buffers are independent:

```
connect requester.mem -> responder.mem {
```

```
    request_buffer 4
    response_buffer 2
}
```

Buffered but unaccepted requests are distinct from accepted outstanding requests. The ledger increments on responder-side request transfer and decrements on requester-side response transfer. Reset begins a new protocol epoch. `ordering out_of_order` requires a matching scalar ID field; hierarchical out-of-order composition remains outside the current bounded subset.

8.6 Aggregate protocols and the standard library

The compiler-shipped `std` namespace maps to ordinary `.zhl` files below `stdlib/`. The compiler knows generic aggregate schemas, roles, ownership, hierarchy, and domains; it does not hard-code AXI/APB transactions.

Available source-authored profiles include RegBus, AXI4-Lite, APB, AXI-Stream, Wishbone B4 Classic, bridges, and the CSR target:

```
import std.bus.reg
import std.bus.axi_lite

module AxiCsrTop {
    clock clk
    reset rst
    interface axi : AXI4Lite<32,32>.slave @clk
    inst frontend : AXI4LiteToRegBus<32,32>
    inst csr : RegBusCSRTarget<32,32>
    connect axi -> frontend.axi
    connect frontend.regbus -> csr.regbus
}
```

Aggregate pass-through connects each member according to typed ownership, including reverse ready:

```
input -> output
```

Protocol member properties such as `.transfer`, FIFO observations, and request/response channel events are part of typed semantics. They are not arbitrary user-defined fields and cannot be assigned when read-only.

8.7 Packets, virtual channels, and arbitration

`packet<T>` extends ready/valid with `.last`. Arbiters explicitly select source order, `fixed_priority` or `round_robin`, and `beat` or `packet` grant lifetime. `vc_credit<T,V,C>` maintains independent credit counts per virtual channel.

8.8 CSR blocks

CSR source describes address layout, access policy, reset, and hardware binding once. Supported policies are `rw`, `ro`, `wo`, `w1c`, `pulse`, and `reserved`. Hardware status uses `<-`, commands use `->`, and sticky events make simultaneous hardware/software priority explicit. RTL, JSON, and Markdown are derived from the same typed model.

Status and sticky sources may select exact leaves of a typed aggregate input; the compiler retains and type-checks the member expression rather than deriving a flattened name:

```
struct Perf { cycles : u32 }
struct EngineStatus { busy : bit perf : Perf }

module AggregateStatusBank {
    clock clk reset rst
    in status : EngineStatus
    csr registers @0 {
        STATUS @0 { busy bit @0 ro <- status.busy }
        CYCLES @4 { value u32 @31:0 ro <- status.perf.cycles }
    }
}
```

Aggregate members used by one CSR bank must belong to that bank's clock domain. Command bindings remain scalar output ports.

Omitted bits are implicitly reserved. A storage-independent access event may observe the same bits as their single owning field:

```
module CsrAccessEventExample {
    clock clk reset rst
    in fault_status : bits<32>
    out fault_clear, registered_clear : bits<2>
    csr registers @0 {
        FAULT @0x470 {
            value bits<32> @31:0 ro <- fault_status
            clear bits<2> @1:0 on_write -> fault_clear
            clear_later bits<2> @1:0 on_write post_accept -> registered_clear
        }
    }
}
```

The event is address-qualified by the compiler and owns no field state. Its default `active_transfer` phase is combinational during the qualified access. An explicit `post_accept` phase captures the selected event value at the accepting edge, publishes it for the following cycle, and then clears it when there is no new matching access. This phase does not change CSR readback or the canonical rule that `rdata` is meaningful only for reads. Stored, external-status, and event observations are available through typed child namespaces `state`, `status`, and `events`. JSON and Markdown maps record the selected event phase.

Repeated layouts flatten into one bank and one decoder:

```
module CsrGroupExample {
    clock clk reset rst
    csr group Window { CONTROL @0 { enable bit @0 rw = 0 } }
    csr control @0 { windows : Window[13] @0x210 stride 4 }
}
```

Group counts are bounded to 1..64 and a bank to 256 expanded registers. A logical 64-bit value may retain two independently writable 32-bit software registers:

```
module CsrSplitExample {
    clock clk reset rst
    csr registers @0 {
        INPUT_BASE @0x18 split<32> value u64 rw = 0 order low_first
    }
}
```

The same `split<32>` declaration may be part of a reusable group. Every group instance receives one logical typed projection and two independently writable 32-bit words:

```
module GroupedSplitExample {
    clock clk reset rst
    csr group Window {
        CONTROL @0 { enable bit @0 rw = 0 }
        BASE @4 split<32> value u64 rw = 0 order low_first
    }
    csr registers @0 { windows : Window[2] @0 stride 12 }
}
```

This does not imply byte strobes or an atomic two-word write. Typed command capture and registered RegBus timing are ordinary source modules: `StorageSnapshot<T,reset_value>` in `std.storage.core` and `RegisteredCSRTarget<AW,DW>` in `std.bus.reg`; the compiler has no special case for either module name.

In a multi-clock module the bank declares its owner after its base address:

```
csr control @0x4000_0000 @apb_clk { /* registers */ }
```

All blocks sharing the canonical access ABI must use that one domain. A runtime CSR value consumed by another domain still requires explicit CDC.

8.9 Clock-domain crossings

Implicit CDC is an error. Existing explicit forms are:

Crossing	Supported endpoint
`sync_level`	Slowly changing `bit` level
`pulse_toggle`	Rate-limited `bit` pulse
`handshake`	One scalar ready/valid payload
`async_fifo(N)`	Ready/valid stream through a power-of-two dual-clock FIFO

```
connect source -> destination { crossing async_fifo(4) }
```

The crossing is a semantic boundary. Its output has destination-domain provenance and may feed destination-domain combinational logic, rules, FSMs, or state normally. The compiler never rewrites, balances, or resource-packs logic through the boundary, and never chooses a crossing automatically.

```
connect enable_a -> enable_b { crossing sync_level }
rule Observe @clk_b when enable_b { seen <- 1 }
```

An aggregate with exactly one forward ready/valid member may use the same `async_fifo` crossing; its complete payload is atomic. Both domains must participate in a coordinated reset episode.

The physical Direct-SV FIFO uses one dedicated 1W1R `async_mem` with a one-cycle registered read, two-stage Gray-pointer synchronizers, and one prefetched output beat. A stalled beat holds both payload and `valid`; the read/consumed pointer advances only on transfer, so that beat still occupies capacity. Continuous accepted beats can transfer at II=1 without an extra output bubble. The synchronizers model digital CDC behavior, not analog metastability or an MTBF guarantee.

The ordinary `std.storage.core.StorageAsyncFifo<T,D>` wrapper exposes this same crossing as a reusable module with explicit writer and reader clock/reset ports. It lowers through the existing CDC IR; its module name is not special to the compiler. A normal `fifo<T,N>` never changes into an asynchronous FIFO because its endpoints happen to use different domains.

No automatic protocol adaptation or implicit CDC is performed. Full AXI4 bursts/IDs and bus-specific CDC bridges remain deferred.

9 Named module interfaces

Named module interfaces give a public name to an exact, behavior-free module signature. They let a library state the ABI that several concrete modules must implement without selecting, replacing, or instantiating any implementation.

```
interface FirIfc {
    clock clk
    reset rst
    in  x : fixed<18,16>
    out y : fixed<18,16>

    timing {
        latency 4
        ii 1
    }
}

module DirectFir : FirIfc {
    // The complete public surface is inherited from FirIfc.
    y = pipeline(4) { x }
}
```

The declaration after `interface` contains only public signature items:

- type and value parameters;
- `clock` and `reset` declarations;
- scalar, wire, ready/valid, credit, packet, and VC-credit `in`/`out` ports;
- source-authored aggregate protocol endpoints and their roles;
- at most one exact `timing { latency N ii 1 }` contract.

It cannot contain assignments, registers, rules, storage, child instances, or other implementation behavior.

9.1 Exact conformance

`module M : Ifc` means that `M` promises to have exactly the applied signature of `Ifc`. Conformance is checked during typed semantic analysis. It is not structural duck typing and does not introduce an implicit conversion.

The following all belong to the contract:

- parameter names, kinds, declaration order, and defaults;
- applied type and value parameter values after specialization;
- port names, declaration order, directions, canonical types, and protocol capacities;
- clock/reset pairs and every port or endpoint domain;
- aggregate protocol specialization, role, member ownership, payload types, and member domains;
- presence and exact value of the module timing contract.

If a conforming module declares no public surface, the complete applied interface surface is inherited: parameters, ports, clock/reset domains, aggregate endpoints, and timing. This is exact signature inheritance, not structural inference. The legacy fully redeclared form remains accepted. A partial redeclaration is never merged with the interface and fails exact conformance.

Missing or additional members in a fully redeclared surface are errors. So are renamed ports, changed directions, width changes, role changes, a different domain, and a timing contract present on only one side.

Named parameters and concrete type parameters are supported:

```
interface TransformIfc<type T, N=4> {
    in  x : vec<N,T>
    out y : vec<N,T>
}

module Transform<type T, N=4> : TransformIfc<T=T, N=N> {
    y = x
}
```

The module parameter declaration itself must match the interface parameter declaration exactly. A concrete specialization records canonical applied values in `ModuleSignature`; concise source spelling does not change that identity.

9.2 Aggregate protocol signatures

An existing source-authored aggregate protocol can be part of a named module interface. Its role and clock domain are explicit:

```
protocol TinyBus<AW=8> {
    role initiator
    role target
    channel command : rv<bits<AW>> initiator -> target
    member alarm : bit target -> initiator
}

interface TinyTarget<AW=8> {
    clock clk
    reset rst
    interface bus : TinyBus<AW=AW>.target @clk
}
```

```
module Target<AW=8> : TinyTarget<AW=AW> {
    clock clk
    reset rst
    interface bus : TinyBus<AW=AW>.target @clk
    // Concrete behavior remains in the module.
}
```

Conformance uses the already typed aggregate schema and ownership model. It does not guess compatibility from endpoint or generated RTL names.

9.3 Current boundaries

The current bounded surface intentionally does not support:

- `request_response` members in named module interfaces, because their requester/responder role is currently inferred from module behavior rather than declared as an exact signature role;
- interface extension or refinement beyond exact complete-surface inheritance;
- multiple-interface conformance;
- automatic implementation selection or substitution;
- relaxed variance, implicit resizing, protocol adaptation, or CDC insertion.

Use the existing concrete module and protocol forms when one of these boundaries applies. Unsupported or non-conforming signatures fail with a structured interface diagnostic; they never silently weaken the ABI.

10 Tagged unions

ZLang tagged unions are a bounded way to carry one of several named scalar payload shapes. They are values, not procedural control flow and not protocol envelopes.

```
union Message {
    Idle
    Data { value : u8 }
    Error { code : bits<4> }
}

message : Message = Message.Data { value = input }

result : u8 = match message {
    Message.Idle => 0
    Message.Data { value } => value
    Message.Error { code } => extend<8>(code)
}
```

10.1 Type and layout

The type is nominal: two equal-looking declarations are not interchangeable. Variant order defines ordinal tag codes. The tag width is `max(1, ceil_log2(variant_count))` and occupies the most-significant bits. The payload is as wide as the largest variant. Fields occupy it from most to least significant in declaration order; a shorter payload has zero low-order padding.

The supported field types are flat `bit`, `bits<N>`, `uN`/`uint<N>`, `sN`/`sint<N>`, `fixed`, and `ufixed` fields. A fieldless value uses the concise constructor `Message.Idle`; `Message.Idle {}` remains an equivalent explicit spelling.

10.2 Construction and matching

Constructor fields are named, exact, and complete. Missing, extra, repeated, or wrongly typed fields are compile-time errors. A `match` must contain every variant exactly once. A payload arm binds every field in declaration order; binders cannot rename, omit, repeat, or shadow an existing value.

Matching is pure and zero-latency. Semantic lowering retains a typed `UnionConstruct`, `UnionTag`, and `UnionField`, then expresses selection with the ordinary typed `Switch`. The simulator carries an immutable nominal runtime

value. Direct-SV carries the exact frozen packed bits. Registers and internal scalar child ports may use union values.

10.3 Deliberate boundaries

An external top-level union input is rejected because arbitrary raw bits could encode an unused tag. There is no raw decode, `bitcast<Union>`, `unpack<Union>`, generic/recursive union, nested aggregate payload, union operator overload, wildcard/nested pattern, guard, partial match, or protocol inference in this slice. This does not change the value's arithmetic meaning.

The runnable source is `examples/tagged_union.zhl`. The exact representation and current exclusions are documented above and in Known limitations.

11 Parameterized aggregate protocols

The generic composition slice expands a parameterized protocol endpoint during semantic elaboration. For example:

```
protocol TinyBus<AW=8> {
    role initiator
    role target
    channel req : rv<uint<AW>> initiator -> target
    channel rsp : rv<uint<AW>> target -> initiator
    member irq : bit target -> initiator
}
```

A module uses this declaration with `interface bus : TinyBus<AW=8>.initiator`. The compiler retains the aggregate identity and specialization, then expands each member into the existing typed leaf endpoint IR. Ready/valid members preserve physical backward-ready propagation; plain members are ordinary scalar wires.

Top-level aggregate endpoints additionally have a stable `TopAggregateABI` projection. It recursively flattens ready/valid payload structs, derives physical direction from source/sink ownership, and keeps aggregate/member identities separate from generated names. Direct-SV consumes that projection without protocol-specific source-name guessing.

`connect left.bus -> right.bus` checks protocol identity, specialization, member names, payload types, roles, and clock domains before producing leaf hierarchical connections. Aggregate buffering, adapters, and CDC are deliberately rejected; they must be expressed on a leaf connection in a later library design.

The Direct-SV backend consumes the same closed child component ABI as other hierarchical protocol children. A child receives every scalar dependency and protocol backward signal explicitly and returns forward values plus scalar outputs. No aggregate or RTL name is reconstructed by textual substitution. Generated interface metadata records both aggregate identities and their leaf signal bindings, so formal/source attribution remains stable.

Generic value parameters support exact positive width arithmetic (`+`, `-`, `*`, and exact `/`). Type parameters are bound at an aggregate use site. The aggregate connection is structural: it does not implement AXI/APB transaction state machines, adapters, packages, CDC, or automatic protocol conversion.

The TinyBus producer/consumer example (two ready/valid channels plus a reverse scalar member) elaborates into three typed leaf connections and passes the release-tested Direct-SV lint gate.

Stateful children with multiple independent ready/valid channels are supported in Direct-SV. An unconnected aggregate bus on the selected top is projected through the shared `TopPhysicalABI` into typed public leaves. This is generic schema/ownership lowering, not an AXI semantic exception. Arrays, partial aggregate exposure, and unsupported protocol kinds are rejected.

12 Standard buses

The production bus profiles are ordinary ZLang sources. The compiler contains no AHB, AXI, APB, AXI-Stream, Wishbone, or RegBus transaction dispatcher.

Import	Source-owned declarations	Initial profile
`std.bus.reg`	`RegBus`, CSR bank/target	In-order request/response CSR boundary
`std.bus.axi_lite`	`AXI4Lite`, `AXI4LiteToRegBus`	32/32-compatible, independent AW/W buffering
`std.bus.axi_burst`	`AXI4BurstSubset`, read/write views and helpers	No-ID, single-outstanding, full-width incrementing bursts
`std.bus.axi4`	Five-channel `AXI4`/`AXI4WithUser`, bounded read/write managers	IDs, bursts and optional USER payloads; under validation
`std.bus.axi4_subordinate`	Bounded read/write subordinate adapters	Backend transactions and responses; under validation
`std.bus.axi4_pins`	Flat-pin adapters	Source-owned five-channel pin projection
`std.bus.axi4_exclusive`	Reservation monitor and exclusive subordinate	Separate explicit same-edge commit contract; under validation
`std.bus.apb`	`APB`, `APBToRegBus`	APB setup/access sequencing
`std.bus.ahb_lite`	`AHBLite`, `AHBLiteToRegBus`	Single-manager, full-width AHB-Lite to RegBus
`std.bus.axi_stream`	`AXIStream`, `AXIStreamPipe`	Data/keep/strb/last ready/valid stream
`std.bus.wishbone`	`Wishbone`, `WishboneToRegBus`	B4 Classic single-beat, ack/err/stall

Error completion is part of the source profile rather than a backend policy. AXI4-Lite maps a failed RegBus write or read to `SLVERR` (`2'b10`) and holds the complete B/R payload while the corresponding ready signal is low. APB forwards the held RegBus error through `PSLVERR` on the access completion. Wishbone uses mutually exclusive normal `ACK` and abnormal `ERR` termination; either one retires the single outstanding request.

AHB-Lite preserves the protocol's pipelined address/data relationship. The bridge captures an accepted address/control phase, consumes write data in the following data phase, and holds `HREADYOUT` low while its one RegBus request is pending. Local size/alignment errors and RegBus failures both use the standard two-cycle ERROR response: low-ready/high-response followed by high-ready/high-response. The first profile accepts aligned, full-bus-width transfers for power-of-two byte-addressable data widths from 8 through 1024; subword transfer strobes are deliberately deferred.

Reusable stateful cores in `std.bus.reg`, `std.bus.ahb_lite`, `std.bus.axi_lite`, `std.bus.axi4`, `std.bus.axi4_subordinate`, `std.bus.axi4_user_subordinate`, `std.bus.axi_burst`, `std.bus.axi_stream`, `std.bus.apb`, and `std.bus.wishbone` do not impose a fixed physical reset spelling. Each inherits the enclosing module's one exact clock/reset contract. Native simulation, formal verification, and generated SystemVerilog all use that same contract. Source-owned flat AXI pin adapters and concrete application tops declare their physical domains explicitly rather than hiding reset policy inside reusable cores.

The runnable AHB example uses conventional active-low `hresetn` with asynchronous assertion and the language's two-edge synchronized release; the same bridge also composes with a synchronous active-high parent. The APB, AXI4-Lite, Wishbone, AXI-Stream, RegBus, and bounded AXI4 witnesses exercise the same inheritance rule. Ambiguous multi-domain inheritance, a stateful core compiled as a direct top without an explicit tooling-owned root domain, and mismatched connected sequential logic fail closed. In every supported AHB contract, `HREADYOUT` is high while reset is asserted.

`std` is a logical namespace mapped to the physical `stdlib/` source tree. Its `.zhl` modules are discovered by convention. Cyclic imports and unsafe paths are rejected.

The AXI4 source modules include IDs, bounded outstanding slots, five independent channels, and optional USER sidebands. Directed and randomized tests cover stalls, repeated IDs, slot reuse, error responses, native simulation against generated Direct-SV, and direct-SystemVerilog generation. These checks do **not** establish complete external endpoint compliance: bounded formal endpoint proofs remain outstanding. The ordinary subordinate never emits `EXOKAY`; exclusive commit requires the separate `axi4_exclusive` contract and same-edge `exclusive_grant` gating. The narrower `axi_burst` profile remains distinct and unchanged. AXI-Stream ID/dest/user, Wishbone burst/retry, automatic adapters, and CDC are not added by these profiles.

Runnable integrated examples are AXI4-Lite CSR, APB CSR, AHB-Lite CSR, Wishbone CSR, and the streaming packet engine. The bounded burst helpers are exposed by the ZTPU AXI burst witness. Python models in `zlang/standard_bus.py` remain independent test oracles only.

The AHB-Lite contract follows the Arm AMBA 3 AHB-Lite protocol rather than the older ZTPU model's simplified same-cycle "AHB-like" behavior. In particular, write address/control and `HWDATA` are not sampled in the same phase, and ERROR is not shortened to one cycle. The source profile and executable tests freeze that bounded contract; broader AHB features remain explicit exclusions below.

12.1 Bounded ZTPU AXI burst subset

`std.bus.axi_burst` is ordinary source-authoritative ZLang HDL. It defines a combined `AXI4BurstSubset<AW,DW>` with independent AR/R and AW/W/B ready/valid channels, together with read-only and write-only protocol views for ZTPU's separate physical master ports. Address payloads carry byte address, eight-bit beats-minus-one `len`, and three-bit beat `size`; read and write data carry `last`, and R/B carry the exact two-bit response value.

The reusable `AXI4BurstReader<AW,DW,LW>` and `AXI4BurstWriter<AW,DW,LW>` accept aligned 1-256-beat requests and permit one outstanding transaction each. AR and AW payloads are snapshotted and remain stable until accepted. Read consumer backpressure drives RREADY directly. The writer accepts AW before exposing W, while AW, W, and B remain independent handshakes; its ready/valid producer retains W data while stalled. Counted framing owns completion: the reader reports early or missing RLAST but drains the requested beat count, while the writer generates WLAST on its locally counted final beat. Any nonzero RRESP or BRESP sets a sticky boolean error for that transaction epoch.

The concrete `AW=64,DW=32` reader/writer witness passes semantic and canonical round trips, deterministic artifact checks, bounded simulator traces, and Direct-SV/Verilator. Validation includes 1- and 256-beat requests, invalid 0 and 257 lengths, misalignment, independent channel stalls, stable owned payloads, RLAST/RRESP/BRESP failures, reset in active phases, and ignored starts while busy. No compiler, simulator, or backend dispatches on AXI names.

This profile deliberately omits IDs, WSTRB, burst-kind, lock, cache, protection, QoS, region and user fields, multiple outstanding transactions, UB-DMA burst chunking, command-descriptor decoding, fences, CDC, or implicit adaptation. Its accepted semantic boundary is the bounded profile stated above.

12.2 AXI4-Lite, APB, and RegBus

`std.bus.reg`, `std.bus.axi_lite`, and `std.bus.apb` are ordinary imported library sources. The compiler checks structural protocol roles, typed ready/valid members, ownership, and domains; transaction state machines belong to library modules. `RegBus<32,32>` is the typed CSR boundary.

The AXI4-Lite and APB profiles use one clock/reset and one outstanding operation. `Axi4LiteToRegBus` buffers AW and W independently and joins them after both transfers; AR returns one held R response. `ApbToRegBus` implements setup/access phases and holds controls stable while waiting for PREADY. Top-level aggregate endpoints expose typed public leaves (for example, `axi_aw_valid` and `apb_psel`); `connect axi -> frontend.axi` delegates a same-role endpoint to a child.

`RegBusCSRTarget` is ordinary source-authored state with RW, sticky W1C, and one-cycle pulse registers. Its response data and valid bit are held until a response transfer. Formal attachment requires exact published observations: properties needing hidden or unsupported state are explicitly skipped or rejected under the selected policy. Full AXI4 is outside these library profiles; the separate bounded AXI burst subset is described above.

13 Standard library

`std` is the compiler-shipped logical namespace. It maps to tracked ordinary ZLang HDL sources under `stdlib/`; it is intentionally shorter than the physical directory name and is not a Python or user-package import.

Import	Purpose
`std.math.fixed`	Pure exact-width/inferred fixed-point abs, clamp, min/max, MAC and explicit quantization helpers, plus compatibility components
`std.math.complex`	Generic `Complex<T>`/`Butterfly<S,D>`, nominal operators, explicit complex quantization
`std.math.complex_fixed_18_16`	Compatibility Q2.16 complex multiply profile
`std.stream.core`	Generic `FrameBeat<T,M>`, ready/valid register slice, skid buffer, FIFO wrapper, and explicit two-way mux/demux

Import	Purpose
`std.stream.serialization`	Bounded power-of-two vector serializer and raw bit collector
`std.stream.complex_fixed`	Compatibility Q5.32-to-Q2.16 complex stream quantizer
`std.dsp.fft`	Exact FFT butterfly, bit reversal, and compile-time twiddle helpers
`std.storage` / `std.storage.core`	Bounded delay/FIFO plus raw-bit reorder and ordered ping-pong storage wrappers
`std.coding` / `std.coding.core`	Parity, bit reversal, checked polynomial-tap LFSR step, and exact convolution helpers
`std.bus.reg`	RegBus and the source-authoritative CSR target
`std.bus.axi_lite`	AXI4-Lite protocol and RegBus frontend
`std.bus.axi_burst`	Bounded no-ID AXI burst profile, read/write views, and single-outstanding burst/single-beat helpers
`std.bus.apb`	APB protocol and RegBus frontend
`std.bus.ahb_lite`	Standards-correct bounded AHB-Lite protocol and RegBus frontend
`std.bus.axi_stream`	AXI4-Stream beat/profile and backpressure-preserving pipe
`std.bus.wishbone`	Wishbone B4 Classic and RegBus frontend
`std.target.generic`	Resource-free generic target identity
`std.target.asic.generic`	Generic ASIC cell/resource capability profile
`std.target.asic.sky130`	SKY130 `sky130_fd_sc_hd` standard-cell synthesis profile
`std.target.intel.cyclone_v`	Bounded Cyclone-V target/resource inventory
`std.target.xilinx.series7`	Bounded source-described Series-7 DSP48E1 profile
`std.target.xilinx.xc7z030`	XC7Z030 part and bounded resource inventory
`std.arch.xilinx7_fir`	Manual four-resource symmetric-FIR cascade template
`std.arch.xilinx7_memory`	Bounded Series-7 storage/resource mapping descriptions
`std.arch.xilinx7_multiply`	Series-7 hard-multiplier architecture descriptions
`std.arch.xilinx7_signed_product`	Signed product-reduction architecture descriptions

Imports are resolved transitively and cycles are rejected. External path/Git packages use the pinned `zlang.toml`/`zlang.lock` project resolver. Filesystem-relative source imports and implicit network lookup are not part of the `std` resolver, and ordinary compilation never fetches dependencies.

Reusable stateful modules across the standard-bus packages inherit their enclosing module's one exact clock/reset contract. Concrete application tops and the source-authored AXI flat-pin adapters own external clock/reset pins. The inherited contract is preserved in specialization identity, simulation, formal products, and Direct-SV; absent or ambiguous direct-top ownership and incompatible hierarchy fail closed rather than selecting a default reset.

`std.bus.axi_burst` is the source-owned ZTPU interoperability profile, not a compiler-recognized bus. It publishes the combined five-channel `AXI4BurstSubset<AW,DW>` plus read-only and write-only views matching ZTPU's separate physical masters. `AXI4BurstReader` and `AXI4BurstWriter` implement one full-width incrementing transaction at a time, with 1-256-beat counting, independent channel backpressure, checked `RLAST`, counted `WLAST`, and deterministic boolean error latching for nonzero `RRESP`/`BRESP`. The validated `AW=64,DW=32` witness passes semantic/canonical restoration, deterministic backend emission, simulator traces, and Direct-SV/Verilator. IDs, write strobes, burst-kind and other full-AXI sidebands, multiple outstanding transactions, UB-DMA chunking, and fences remain outside this bounded profile.

For one full-width beat, `axi_single_beat_address<AW,DW>` constructs `AxiBurstAddress<AW>` with `len=0` and the matching `size`, plus a validity bit for geometry and alignment. `AXI4SingleBeatReader` preserves the burst reader's one-beat RLAST/RRESP checks; `AXI4SingleBeatWriter` snapshots AW and W payloads and allows either channel to transfer first. Both retain the one-outstanding `busy`/`done`/`error` convention. See `examples/axi_single_beat.zhl`. These helpers do not add IDs, WSTRB, or implicit bus adaptation.

`std.bus.ahb_lite` follows the AHB-Lite address/data pipeline and two-cycle ERROR response rather than the historical ZTPU model's same-cycle AHB-like shortcut. `AHBLiteToRegBus<AW,DW>` accepts one aligned full-width beat at a time for byte-addressable power-of-two `DW` from 8 through 1024, supplies an all-byte RegBus write mask, and holds the AHB data phase through RegBus request/response stalls. Subword transfers, arbitration, SPLIT/RETRY and compiler/backend AHB dispatch are not part of this profile. Its `hresetn` example domain uses active-low asynchronous assertion and two-edge synchronized release. The bridge itself inherits the enclosing module's one exact physical clock/reset contract, so the same source also composes with a synchronous active-high system. Ambiguous multi-domain inheritance and mismatched connected state owners remain errors; this is not an implicit reset or CDC conversion.

std.math.complex is ordinary source-authoritative ZLang. It has no bus or stream dependency and uses only generic structs/functions and nominal operator declarations; semantic analysis and the Direct-SV backend have no Complex special case. Mixed fixed-point multiplication retains its full exact width and scale, and the caller places every quantization boundary explicitly. In particular, the core contains no implicit fixed<18,16> butterfly. The historical Q2.16 component and stream profiles live in separate compatibility imports std.math.complex_fixed_18_16 and std.stream.complex_fixed; importing numerical Complex arithmetic does not select either profile or import a bus.

13.1 Fixed-point math

std.math.fixed provides pure generic fixed_abs, fixed_min, fixed_max, fixed_clamp, and exact fixed_mac functions plus explicit rounding/overflow helpers such as fixed_quantize_nearest_even_saturate<T>. fixed_abs widens according to ordinary unary-negation rules, so the most-negative input has an exact positive result; fixed_mac preserves the full product and addition widths. The library also retains the source-authored FixedAbs, FixedClamp, FixedMinMax, FixedMAC, FixedSaturatingAdd, and UFixedSaturatingAdd components. They operate on canonical fixed-point types; concise SF8.8, SF_Sat8.8, UF8.8, and UF_Sat8.8 forms are language aliases rather than library implementations. Encoding and conversion rules are specified in fixed-point-types.md.

13.2 Complex values, streams, storage, and coding

std.math.complex provides the ordinary parameterized value type Complex<T> with re and im fields. butterfly_quantized<T> requires the output numeric profile explicitly; there is no profile-selecting short spelling in the core. Complex remains ordinary source, not a compiler or backend primitive.

std.bus.axi_stream keeps AXIStream<DW> as the byte-lane bus profile and also provides AXIStreamOf<T> for a semantic payload type:

```
input  : AXIStreamOf<Complex<fixed<18,16>>>.sink @clk
output : AXIStreamOf<Complex<fixed<18,16>>>.source @clk
```

AXIStream<32> has data/keep/strb/last; AXIStreamOf<T> transports exactly one typed T per transfer. It does not by itself specify an external AXI TDATA serialization for arbitrary T.

Generic stream composition uses std.stream.core, independently of AXI:

```
import std.stream.core

struct Meta { tag : u2 }
type Beat = FrameBeat<u8,Meta>

module Queue {
    clock clk
    reset rst
    in input : rv<Beat>
    out output : rv<Beat>

    inst storage : RvFifo<T=Beat,D=4>
    connect input -> storage.input
    connect storage.output -> output
}
```

RvRegisterSlice<T>, RvSkidBuffer<T>, and RvFifo<T,D> reuse the exact language FIFO semantics, including simultaneous push/pop and payload stability under stall. RvMux2<T> and RvDemux2<T> provide explicit runtime routing between statically elaborated ready/valid endpoints. They route backpressure only through the selected path, never create dynamic instances or clock gates, and can be composed as a bounded tree for wider routing. Scalar routing remains ordinary mux and explicit valid/enable gating. std.stream.serialization currently supports a power-of-two RvVectorSerializer<T,N,IW> and a raw RvBitCollector<N,IW>, where IW = floor_log2(N). Their declarations enforce N >= 2, power-of-two N, and exact IW through compile-time where constraints.

std.dsp.fft keeps arithmetic intent explicit. Its butterfly does not quantize, twiddle generation uses compile-time sin/cos and quantizes only to its caller-selected type, and bit reversal is a deterministic sequence-order operation. Value parameters inferred through a bits<N> argument are not accepted, so bit-width helpers use explicit calls such as fft_bit_reverse<N=8>(value).

`std.storage.core` supplies `StorageDelay1<T>`, `StorageDelay2<T>`, the generic `StorageQueue<T,D,CW>` (`CW = ceil_log2(D + 1)`), and bounded raw-bit `StorageReorderBits<W,N>`/`StoragePingPongBits<W,N>` banks. The latter use power-of-two depth, exact runtime indices, and explicit bitcast at typed/raw boundaries. Ping-pong publication is ordered: a second `commit` is blocked until the visible read bank retires, while simultaneous retire/commit is legal. The same file provides ordinary `StorageDualPortMemory<T,N,AW>`, `Storage2R1WMemory<T,N,AW>`, and `StorageAsyncMemory1W1R<T,N,AW>` wrappers over the shared typed memory IR. `StorageAsyncFifo<T,D>` wraps the existing explicit ready/valid `async_fifo(D)` crossing. These module names are not compiler intrinsics and do not authorize implicit CDC. It also provides immutable, source-authored generic ROM wrappers:

```
inst direct : StorageRom<T=u8,N=8,IW=3,image=image>
inst generated : StorageGeneratedRom<
    T=u8,N=8,IW=3,producer=fn make_image<T=u8,N=8>
>
```

Both wrappers elaborate to the existing typed `Rom` IR with concrete immutable contents and the deterministic companion image used by Direct-SV. `StorageRom` accepts a fully evaluated exact `vec<N,T>` constant; `StorageGeneratedRom` invokes a statically selected pure zero-argument producer during elaboration. Constants and producers are specialization parameters, not runtime ports or backend callbacks. Fixed, struct, vector, and nested-vector word types retain their exact canonical layout.

The core language currently requires a literal in `delay<N>`, so a generic-depth delay wrapper would be dishonest; longer delays remain explicit source until parameterized delay depth is supported. `std.coding.core` supplies representation-level parity, bit reversal and one-step LFSR operations plus exact dot/convolution and `table_gather` helpers. `table_gather<T,N,IW>` is accepted only when the complete proven `uint<IW>` range fits the source vector. It preserves order but deliberately does not claim bijection: duplicate and omitted elements remain legal. The `CodingLfsrStep<N>` module enforces `N >= 2`; the compatibility pure function cannot carry a `where` clause because function constraints are not yet syntax.

The current language deliberately prevents several tempting but invalid ”generic” wrappers. A target-independent `reg vec<N,T>` requires an explicit initializer; `default<T>` is not supported. The shipped reusable reorder/ping-pong banks therefore use an explicit `bits<W>` representation boundary. A generic runtime gather now retains the conservative element-type range through a table-loaded index, but it does not prove that a table is a mathematical permutation. The negative semantic tests retain the bounded gather witness; no backend guesses an initializer or claims a permutation proof. Concrete projects may still use typed ROMs and permutations with known source-generated tables, as the Wi-Fi and FFT sources do.

The separated `std.stream.complex_fixed` compatibility kernel exposes raw `rv<Complex<...>>` ports. The former aggregate AXIStream profile cannot be forwarded transparently yet: hierarchical connections between `AXIStreamOf<Complex<...>>` specializations and member-level child ready/valid ports are rejected by the current aggregate binding model. The kernel is fully backend-validated, but restoring that aggregate ABI requires a generic aggregate-member binding compiler slice rather than bus-specific stdlib code.

Every shipped `.zhl` file is discovered recursively by the `std.*` resolver. Clean-wheel tests compare the complete recursive source tree with wheel contents; adding a library file without packaging it therefore fails the build rather than creating a checkout-only import.

14 Projects and dependencies

ZLang source imports are logical names. Physical paths, Git URLs, and revisions belong to project metadata, never to `.zhl` source:

```
import acme.dsp.filters
import std.math.complex
```

An import may introduce a source-local qualifier without changing the logical module or dependency identity:

```
import std.math.complex as cx

in sample : cx.Complex<u8>
out total : cx.Complex<u9>
total = cx.complex_sum([sample, sample])
```

The qualifier applies to types, functions, and struct constructors declared by that exact logical module (`cx.Complex { re = ... im = ... }` is valid). It is erased before semantic typing, so the qualified and historical unqualified spellings have the same canonical identity. An aliased import does not expose those declaration names unqualified and does not re-export declarations from a transitive dependency. Aliases are not runtime values or filesystem names; wildcards, member renaming, and re-export remain unsupported.

The compiler-shipped `std` namespace keeps its existing resolver. Other package names are resolved only when the source belongs to a locked project.

14.1 Project manifest

A project is rooted by a versioned `zlang.toml`:

```
schema = 1

[project]
name = "demo"
version = "0.1.0"
source-root = "src"

[dependencies]
acme = { path = "../acme-dsp" }

[dependencies.bus_models]
git = "https://example.invalid/hardware/bus-models.git"
rev = "0123456789abcdef0123456789abcdef01234567"
```

The `schema` field is required project metadata: it tells `zlang lock update` how to interpret the manifest. Use the value shown for the current project format.

Imports use logical package and module names rather than absolute checkout or cache paths. A file `filters.zhl` directly below package `acme`'s source root is imported as `acme.filters`; nested directories append dotted components.

Path locators are relative to the manifest containing them. Git revisions must be full lowercase 40- or 64-digit hexadecimal object IDs. Branch names, tags, and abbreviated revisions are deliberately rejected because they are mutable.

14.2 Lock update

Dependency resolution and fetching are explicit:

```
zlang lock update --project zlang.toml
```

The command validates the complete transitive graph before publishing a deterministic `zlang.lock`. It records each package manifest, exact source-module index, imports, content digests, and dependency edges. Git content is populated in the project cache during this command. Path dependencies remain at their manifest-relative location and are checked byte-for-byte against the lock.

Resolution rejects:

- dependency cycles and conflicting package/module declarations;
- source-root traversal or symlink escape;
- a package whose declared identity does not match its dependency key;
- missing, added, removed, or modified locked `.zhl` modules;
- modified dependency-resolution fields in dependency manifests (profile-only edits are intentionally outside the resolution identity);
- missing Git cache content or a revision mismatch.

The lock is written only after the entire graph validates. Failed updates leave the previously accepted lock usable.

Scenario	What to do	Without it
Standalone source or compiler-shipped `std.*` imports	Compile directly; no project lock is needed.	No consequence.
Project with local path packages	Run `zlang lock update` after declaring dependencies and whenever their source or resolution fields change. Keep `zlang.lock` with the project.	Compilation rejects a missing or stale lock; it never silently uses changed dependency files.
Project with pinned Git packages	Run `zlang lock update` while Git/network access is available; retain the exact revision and local cache for later offline builds.	Ordinary compilation does not fetch; missing cache or changed revision fails.
CI or another machine	Supply the project manifest, its lock, exact path sources and cached Git revisions (or update the lock before the offline build).	A lock file alone does not provide Git source bytes; compilation fails closed.

`zlang lock update` resolves dependencies; it is not a package installer or a step required for every single-file invocation.

Bounded scalar `extern module` implementations may also be declared under `[external-mappings.NAME]` and selected by a profile's `external-mappings = ["NAME"]`. The lock records every HDL source path and SHA-256. Paths are project-relative, must remain inside the project, and may not be symlinks. The mapping changes generated SystemVerilog without changing the behavior expressed by the ZLang source.

14.3 Offline compilation

Ordinary compilation never fetches dependencies and never updates project metadata:

```
zlang src/top.zhl --project zlang.toml --check
zlang src/top.zhl --project zlang.toml --systemverilog build/top.sv
```

`--project` may name a manifest or its directory. Without it, `zlang` searches the source file's parent directories for `zlang.toml`. If no project is found, single-file and compiler-shipped `std.*` compilation retain their existing behavior; arbitrary external imports remain an explicit error.

Compilation checks the current manifest, lock, every path dependency, and every cached Git dependency before checking the source. It is read-only: unavailable or dirty content is a lock mismatch, not an invitation to fetch or rewrite files. Build outputs and ZLang output/cache directories must remain outside the resolved root, manifest, lock, dependency, and stdlib inputs.

14.4 Deliberate boundaries

The current project model has no registry or semantic-version solver, editable global packages, implicit network access, Git submodules/LFS/subdirectories, wildcard imports, member renaming, or re-export. Implementation profiles are a separate set of build settings and do not change dependency resolution. Path dependencies declared by a Git package are also deferred in this bounded slice; use another pinned Git dependency instead of reaching outside a fetched checkout.

Ordinary compilation remains offline; resolved module contents and their recorded digests define the accepted dependency contents.

15 Implementation intent, frequency, and II

Use `implement` when an expression defines one exact hardware value but more than one legal implementation may exist. The expression remains the required result. The nested `intent` block filters legal choices and optionally states which cost to prefer:

```
module IntentDotProduct {
    clock clk reset rst
```

```
    in a : vec<4,u3>
    in b : vec<4,u3>
    out y : u8

    y = implement {
        dot(a, b)
        intent {
            latency <= 3
            ii == 1
            dsp <= 4
            fmax >= 100
            minimize latency
        }
    }
}
```

The compiler keeps the original exact expression as a legal choice and may also consider a bounded set of equivalent structures and physical implementations. It never relaxes a hard constraint. If no candidate satisfies every constraint, compilation fails and reports the rejected choices.

15.1 `intent` clauses

An `intent` block is non-empty. Clauses are separated by whitespace or newlines; commas are not used. A metric may have at most one constraint, and the block may have at most one objective.

Form	Accepted values	Meaning
`lut REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the LUT estimate.
`ff REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the flip-flop estimate.
`dsp REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the DSP-resource estimate.
`bram REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the block-memory estimate.
`latency REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound accepted-input to corresponding-output latency according to the selected contract. A positive value requires a clock/reset context.
`ii REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 1`	Bound initiation interval: the nominal minimum spacing between accepted transactions when downstream capacity is available.
`fmax REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 1` MHz	Bound the available frequency estimate or compatible measurement. `fmax_est` is accepted as an alias.
`minimize METRIC`	`lut`, `ff`, `dsp`, `bram`, or `latency`	Rank legal choices by one cost.
`maximize fmax`	`fmax` or `fmax_est`	Prefer the highest compatible frequency evidence.

If an objective is omitted, selection defaults to `minimize lut`. Frequency may only be maximized; resources and latency may only be minimized. II has no objective form. Constraints are filters, not requests to invent a new architecture: for example, `ii <= 4` may select an existing II=1 choice.

`pipeline(N) { expression }` is different from `implement`. It always specifies exactly `N` cycles of visible latency. An `implement` block instead allows the compiler to select among the existing legal choices admitted by its constraints. Use `choice(...)` when the source explicitly supplies alternative expressions.

15.2 Frequency control and synthesis trials

Frequency selection and the physical synthesis clock are separate controls:

Control	Purpose
`fmax >= 100` inside `intent`	Reject an implementation choice whose admitted frequency value is below 100 MHz.
`maximize fmax`	Rank the remaining legal choices by their admitted frequency values.
`fmax = { minimum = 100 }` in a selected project profile	Apply the same minimum to profile-selected regions.
`period-ns = 10.0` under the profile clock table	Emit a real 10 ns XDC/SDC clock constraint for downstream synthesis and place-and-route.
`evidence-policy`	Decide whether estimates are accepted, measurements are preferred, or measurements are required.

For a target of `F` MHz, the corresponding physical period is `1000 / F` ns. Setting only `fmax >= F` does not create a clock constraint and does not prove timing closure. Conversely, a clock period constrains the downstream tool but does not guarantee that the design meets it.

A project profile can keep these settings together:

```
[profiles.synth_trial]
backend = "systemverilog"
backend-mode = "required"
target = "xc7z030ffg676-1"
objective = "maximize fmax"
evidence-policy = "measured_preferred"
formal-policy = "available"

[profiles.synth_trial.constraints]
latency = { maximum = 4 }
ii = { exact = 1 }
fmax = { minimum = 100 }
dsp = { maximum = 8 }

[profiles.synth_trial.platform.clocks.clk]
period-ns = 10.0
```

Emit SystemVerilog and matching constraints from that profile:

```
zlang src/top.zhl --project zlang.toml --profile synth_trial \
  --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc \
  --constraints-sdc build/Top.sdc \
  --exploration-report build/implementation.txt
```

Supply `Top.sv` and the appropriate constraint file to the FPGA/ASIC synthesis flow. ZLang does not silently launch a vendor tool. For a bounded generic Yosys characterization trial, request both a report and cache:

```
zlang src/top.zhl --project zlang.toml --profile synth_trial \
  --systemverilog build/Top.sv \
  --synthesis-report build/yosys-synthesis.txt \
  --synthesis-cache build/yosys-cache \
  --synthesis-target generic-lut6
```

`estimate_only` uses estimates. `measured_preferred` ranks compatible routed measurements ahead of synthesis measurements and estimates, but still permits an estimate when no compatible measurement exists. `measured_required` rejects a choice when the required compatible measurement is absent. Measurements are specific to their target, part, tool, recipe, constraints, and design; a result from a different trial is not silently reused.

15.3 Ready/valid contracts and II

For a ready/valid boundary, transactions occur only on transfers:

```
input transfer  = input.valid  && input.ready
output transfer = output.valid && output.ready
```

Latency is the unstalled distance from an accepted input to its corresponding valid output. II is the nominal minimum spacing between accepted inputs when the implementation and downstream path have capacity. Capacity is the number of accepted but not yet retired transactions the block can hold. Downstream backpressure may increase the observed wall-clock interval; it does not change the declared II.

Current support has three distinct cases:

Form	Current behavior
Ordinary scalar `implement`	Fixed-rate only. Available candidates have II=1; an II>1 constraint does not create resource sharing.
`transform pipeline(auto, ...)` over a supported ready/valid kernel	Globally stalled pipeline with unstalled II=1. Output data and `valid` hold while downstream `ready` is low.
`transform pipeline(auto) { implement { a*b + c*d ... } }`	Narrow non-interleaved shared arithmetic: one multiplier, latency 4, II 4, capacity 1.

The supported II=4 form requires one ready/valid input and output, integer operands, the exact `a*b + c*d` shape, one clock/reset domain, `dsp <= 1`, and an intent that admits II 4 and latency 4:

```
struct SharedOperands {
    a : u16
    b : u16
    c : u16
    d : u16
}

module SharedMultiply {
    clock clk reset rst
    in input : rv<SharedOperands>
    out output : rv<u33>

    input -> output {
        transform pipeline(auto) {
            implement {
                input.payload.a * input.payload.b
                    + input.payload.c * input.payload.d
                intent {
                    latency <= 4
                    ii == 4
                    dsp <= 1
                    minimize dsp
                }
            }
        }
    }
}
```

Only one transaction is active. While computation or an unconsumed result is present, new input is blocked. When `output.valid && output.ready` retires a result, the block may accept the next input on the same edge; this is why the implemented contract is II 4 rather than II 5. A stalled output remains valid with stable payload, and input readiness remains low until retirement is possible.

This is not general HLS. It does not support overlapping transactions, modulo scheduling, arbitrary arithmetic graphs, more products, fixed-point temporal sharing, memories, CDC, or multiple active transactions. The transaction-stream formal route supplies bounded model-checking evidence for this exact case. `required_bmc` may accept it only after `bounded_pass`; `required_proven` rejects it because no unbounded proof is claimed.

The runnable example is `examples/temporal_shared_multiply.zhl`.

16 Formal verification

ZLang HDL supports named same-cycle safety properties and bounded cover goals. Place goals in the source next to the hardware they describe:

```
module CheckedCounter {
    clock clk reset rst
    in enable : bit
    out value : u8

    reg count : u8 = 0
    step: when enable { count <- truncate<8>(count + 1) }
    value = count

    assert never_wraps_early @ clk { (count < 255) | !enable }
    cover reaches_ten @ clk { count == 10 }
}
```

Run all applicable goals for a selected top:

```
zlang design.zhl --top CheckedCounter --verify \
  --formal-jobs 4 \
  --verification-report build/verification.txt \
  --verification-work-dir build/verification-work
```

For a reproducible two-step flow, first publish a verification bundle and then run it:

```
zlang design.zhl --top CheckedCounter \
  --verification-bundle build/verify
zlang verify build/verify --mode bmc --depth 20 \
  --work-dir build/verify-work
```

The work directory contains solver logs and any counterexample or cover VCD. Keep it outside the bundle so reruns do not modify the input being checked.

Result terms have precise meanings:

- `bounded_pass`: no counterexample was found through the stated depth; this is not an unbounded proof;
- `proven`: the selected proof flow completed an unbounded proof;
- `witnessed`: a cover goal was reached and has a witness;
- `bounded_unreached`: a cover was not reached within the selected bound;
- `unknown`, `unavailable`, or `skipped`: no proof claim was established.

Each clocked goal names its clock with `@ clock_name`. Reset behavior follows the module's declared polarity, assertion mode, and release contract. Goals in independent supported domains run separately; this does not create a cross-domain temporal property. Use explicit assumptions only for real environment constraints, and never use an assumption to hide an invalid design.

Formal execution requires Yosys, SymbiYosys, `yosys-smtbmc`, and a supported solver such as Z3. The installation chapter shows how to check those tools. Examples are available in `examples/verification`.

17 Implementation profiles

Implementation profiles keep physical policy outside portable `.zhl` source. They are named tables in `zlang.toml` and are selected explicitly:

```
[profiles.release]
backend = "systemverilog"
backend-mode = "required"
target = "xc7z030ffg676-1"
allowed-transforms = ["pipeline", "dsp", "reduction"]
avoided-transforms = ["reassociate"]
objective = "maximize fmax"
```

```
architecture = "Xilinx7SymmetricDSPCascade"
architecture-mode = "preferred"
evidence-policy = "measured_required"
formal-policy = "required_bmc"

[profiles.release.constraints]
latency = { maximum = 8 }
ii = { exact = 1 }
fmax = { minimum = 100 }
dsp = { maximum = 8 }
```

`maximize fmax` prefers a legal choice with the strongest compatible frequency measurement available under the selected evidence policy. It does not guarantee the frequency achieved on an FPGA or ASIC. An estimate is never reported as measured routed timing.

Schematic use inside a project containing the shown `src/fir.zhl` and profile:

```
zlang src/fir.zhl --profile release --systemverilog build/Fir.sv
```

A profile may select pinned scalar external-module mappings with `external-mappings = ["vendor-add"]`. Mapping declarations live in the top-level `external-mappings` tables of `zlang.toml`; their source files and hashes are fixed by `zlang.lock`. They may change generated SystemVerilog but do not change the source-language behavior of the design.

Only the selected profile is parsed strictly. This permits a project to carry profiles for newer compilers without making unrelated builds fail. Selecting an unknown profile, selecting a profile without a project, or using an unknown key is an error. Profile data is excluded from dependency resolution and lock contents.

18 Platform constraints

18.1 Clock constraints

A selected project profile can declare one explicitly named clock period:

```
[profiles.release.platform.clocks.clk]
period-ns = 10.0
```

The clock name must match the selected top's clock declaration. The period is a physical build setting and is not inferred from an `fmax` intent.

The CLI writes constraints only together with exactly one SystemVerilog output. The following command assumes a project-local `top.zhl` and the `release` profile shown above:

```
zlang top.zhl --profile release --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc --constraints-sdc build/Top.sdc
```

XDC and SDC initially contain only:

```
create_clock -name clk -period 10 [get_ports {clk}]
```

The clock edge and reset mode must match the source declaration. A stale or mismatched constraint request is rejected. XDC/SDC output is deterministic and is written atomically without following symlinks.

No reset false path is emitted: asynchronous-reset recovery/removal timing must not be hidden. This constraint publication supports only the exact single-clock profile shown here. Multiple clock domains are supported in language semantics and Direct-SV emission, but generated clocks, multi-clock XDC/SDC publication, pin/-package/IO properties, input/output delays, CDC constraints, and formal properties are not emitted by this facility.

19 Generate SystemVerilog

Write the selected top to an explicit file:

```
mkdir -p build
zlang design.zhl --top Top --systemverilog build/Top.sv
```

The command writes only the requested artifact and keeps stdout empty. Diagnostics and `--verbose` progress messages go to stderr. A bare `zlang design.zhl --top Top` invocation writes SystemVerilog to stdout, which is convenient for a pipe but less suitable for a reproducible project script.

Use `--check` when no RTL should be created:

```
zlang design.zhl --top Top --check
```

Lint the generated module with Verilator:

```
verilator --lint-only --top-module Top build/Top.sv
```

Project profiles can select a target and clock constraint:

```
zlang top.zhl --profile release \
  --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc \
  --constraints-sdc build/Top.sdc
```

Generated RTL preserves declared widths, signedness, reset modes, hierarchy, and protocol behavior. Unsupported combinations stop with a diagnostic rather than emitting a partial module. Public port names are stable; private helper names are not a source-language interface and should not be referenced by user code.

The former `--experimental-systemverilog` option is not supported. Use `--systemverilog` for file output or the bare command for stdout.

20 Known limitations

ZLang HDL rejects unsupported combinations instead of generating partial RTL. The following restrictions are part of the current user-facing contract.

20.1 Language and RTL boundaries

- ZLang has no runtime procedural `if`, mutable local variables, implicit numeric casts, implicit fixed-point rescaling, or general HLS scheduler.
- Runtime-selected instance inputs/protocols, general cross-module atomic scheduling, full AXI4, automatic CDC insertion, and arbitrary stateful elastic pipelines are unsupported.
- Stateful objects inside one module may belong to independent explicit clock domains. A bounded `async_mem` is the sole storage exception: its one writer and one registered reader have distinct owners and form an explicit semantic boundary. Asynchronous 2RW memory, mixed widths, automatic banking, cross-clock atomic rules, derived/gated clocks, and target-aware scheduling across a CDC boundary are unsupported. Ordinary global memory control cannot be combined with unrelated user register/rule state in the same module; use hierarchy for that composition.
- Unsupported Direct-SV combinations produce a diagnostic rather than partial RTL.

20.2 Verification boundaries

Formal support is specific to each property family and observation boundary. `bounded_pass` is not an unbounded proof, and `skipped`, `unavailable`, and `unknown` make no proof claim. Report any accepted design that emits invalid RTL as a correctness defect.

A ZLang HDL concise source-authoring reference

This page is a concise guide to writing and running ZLang HDL. Source-language rules define the hardware; generated SystemVerilog is an output format, not a second language specification.

For an unfamiliar construct, consult the detailed language chapter and known limitations. For exact width tables and supported forms, follow the linked topic guide.

A.1 Non-negotiable rules

- Source files use `.zhl`. The old `.zl` suffix is rejected.
- Run `zlang SOURCE --check`; highlighting is lexical and is not validation.
- Native simulation needs a matching platform wheel and fails explicitly when it is unavailable.
- Hardware assignments are concurrent. `=` drives the current cycle and `<-` schedules next state at the active clock edge.
- Assignment types are exact. There is no implicit resize, signedness change, fixed-point rescale, protocol adapter, or clock-domain crossing.
- Do not invent `let`, `const`, `var`, `return`, semicolons, C-style `input`/`output`, or runtime procedural `if`.
- `if` is compile-time elaboration only. Use `?:`, `mux`, or `switch` for pure runtime values; use `when`/`else when`/`else`, `priority`, or `fsm` for state actions.
- Scalar `pipeline(auto)`, `architecture(auto)`, and `explore` are retired. Compiler-selected scalar implementation uses `implement`.
- `pipeline(N) { expression }` is exact N-cycle hardware, not an optimization request. Protocol `transform pipeline(auto, ...)` is a separate bounded ready/valid construct and remains supported.
- Unsupported combinations fail with a diagnostic. Do not work around one by guessing generated signal names.

A.2 Smallest valid modules

Combinational logic:

```
module Add {
    in a, b : u8
    out y : u9 = a + b
}
```

Sequential logic:

```
module Accumulator {
    clock clk reset rst
    in enable : bit
    in value : u8
    out total : u9

    reg state : u9 = 0
    update: when enable {
        state <- truncate<9>(state + extend<9>(value))
    }
    total = state
}
```

All operands are read from one pre-edge snapshot. An accepted action group commits atomically; reset has priority. Multiple conflicting actions require an explicit `priority` relation or are rejected.

Multiple domains use the same declarations with one `@clock` owner:

```
module DualClock {
    clock a reset ra @a
    clock b reset rb @b
    reg left : u32 @a = 0
    reg right : u32 @b = 0
    rule TickLeft @a when 1 { left <- truncate<32>(left + 1) }
    rule TickRight @b when 1 { right <- truncate<32>(right + 2) }
}
```

An omitted owner is accepted only when exactly one domain can be inferred. Dynamic provenance follows through local expressions; an explicit `sync_level`, `pulse_toggle`, `handshake`, or `async_fifo(N)` is the only way to cross domains.

A.3 Types, literals, and exact conversions

Common scalar types are `bit`, `u8`, `s16`, `uint<N>`, `sint<N>`, and `bits<N>`. `char` is canonical `u8`; `string<N>` is canonical `vec<N,u8>`. Fixed point uses `fixed<W,F>`, `ufixed<W,F>`, or concise forms such as `SF8.8` and `SF_Sat8.8`.

- Unsuffixed integer literals use minimum exact width unless they occur directly at a concrete typed boundary.
- `a + b` and `a - b` follow the documented exact result-width rules; multiplication returns the full product width. Shifts preserve left width.
- Use `extend<N>` and `truncate<N>` for width changes.
- Use `quantize<T>` for explicit fixed-point rounding/overflow. Do not move a quantization boundary into products or reductions.
- Compile-time `pi`, `sin`, `cos`, `log2`, `log`, `exp`, and `sqrt` generate constants only; non-integral results require explicit `quantize<T>`.
- Use `bitcast<T>` only for equal-width representation changes. `pack`/`unpack` are supported low-level compatibility forms.
- `concat(a,b,...)` places the first operand at the MSB. Destination context never changes operand widths.
- Packed bit index zero is the LSB. `x[MSB:LSB]` uses an inclusive static slice; vector ranges `values[first..past_last]` are half-open.
- Indexed aggregates are packed LSB-first: `vec`/`string` element zero and tuple `item0` occupy the least-significant component recursively. First-declared struct fields and tagged-union tags remain at the MSB.

See types and numerics before changing a width or a fixed-point type.

A.4 Aggregates and functions

```
struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}
```

Function bodies are pure and combinational. Bare body bindings are immutable, inferred, declaration-ordered locals; the final expression is the result. Return types may be explicit or inferred. Functions cannot capture module ports or contain state, protocols, delays, or pipelines.

Supported aggregates include `vec<N,T>`, nominal `struct`, nominal `enum`, tagged `union`, and structural tuples `(T,U)`. Prefer inferred locals when the type is exact and obvious. Keep explicit types at public ports, state, raw/enum boundaries, narrowing, and fixed-point quantization.

Generics are monomorphized. Type/value parameters and exact compile-time constraints are source declarations, not runtime hardware:

```
module Queue<type T,D=4> where D >= 2 && is_power_of_two(D) {
    // ...
}
```

Imports are logical. For example `import std.bus.reg` resolves compiler-shipped `stdlib/bus/reg.zhl`; `import std.math.complex as cx` adds a source-local qualifier but does not create a runtime namespace.

A.5 State, hierarchy, and protocols

- Declare one default synchronous domain with `clock clk reset rst`.
- `async reset arst @clk` means asynchronous assertion and fixed two-edge synchronized release. Do not emulate reset modes in user logic.
- Add `release externally_synchronized` only when deassertion is already synchronized outside; no internal 2FF is emitted.
- `reg q:T = VALUE` resets to `VALUE`; `reg q:T` has no reset branch.
- `out q:T` is a combinational output wire.
- `out reg q:T [@clock] [= VALUE]` is real architectural state: it holds when not updated and follows register clock/reset scheduling. It is not Verilog's historical `output reg` declaration category.
- `drive q = expression` is a transient rule/FSM-owned logical wire effect, zero when no owning action fires. It is not tri-state, high-impedance, electrical-strength, pad, analog, or multiple-driver-resolution behavior.

- `reg`, `fifo`, `memory`, and `rom` retain explicit typed state/storage semantics. Memory latency, collision, mask, and reset policies are not inferred.
- A concise child declaration is `child : ChildModule`; `inst child : ChildModule` remains accepted. Bind only typed ports/endpoints.
- `rv<T>`, `credit<T,N>`, and `request_response` properties such as `.transfer` are typed observations, not ordinary struct fields.
- Use explicit `connect`/`source -> sink`, adapters, and CDC crossings. The compiler never inserts these silently.
- Top-level structs and tuples become recursively named leaf ports; vectors use multidimensional packed arrays in the production Direct-SV ABI.

Read sequential state/storage and hierarchy/protocols before composing stateful children.

A.6 Functional datapath and implementation intent

`generate`, `map`, `reduce`, `sum`, and `dot` are statically bounded functional hardware expressions. They are not runtime loops.

Use exactly one `implement` region when the value is fixed and the compiler may choose among legal implementations:

```
y = implement {
    dot(samples, coefficients)
    intent {
        latency <= 4
        ii == 1
        dsp <= 8
        fmax >= 100
        minimize lut
    }
}
```

An Fmax constraint or objective compares available implementation choices. It is not a guarantee of achieved device or silicon frequency; estimates, synthesis measurements, and routed measurements are distinct.

The expression is the hardware meaning. `intent` contains hard constraints and one supported objective; it does not name optimization passes. Search is bounded and deterministic rather than an exhaustive optimality guarantee. Positive-latency choices require a valid clock/reset context. `choice` is reserved for user-authored equivalent alternatives. Project profiles may set the documented `intent-exploration` count limits; there is no source-level timeout.

See implementation intent, frequency, and II for every accepted metric/objective, synthesis clock control, project-profile configuration, and the exact ready/valid II>1 boundary.

A.7 Verification

```
contract arithmetic @ clk {
    require legal_inputs { (a < 8) & (b < 8) }
    ensure exact_output { y == a + b }
    cover maximum { y == 14 }
}
```

Named `assert`/`ensure` are same-cycle safety goals. `require` is an environment-owned precondition scoped to its contract. `cover` is bounded reachability; a missed cover is not an unreachability proof. BMC produces `bounded_pass`, never `proven`. Verification syntax does not legalize invalid hardware or feed optimizer range inference.

Liveness, arbitrary SVA/SMT, temporal sequences, and source-level equivalence controls are outside this syntax. Start from the existing formal examples.

A.8 Commands and completion check

```
zlang source.zhl --check
zlang source.zhl --top Top --systemverilog build/Top.sv
zlang source.zhl --top Top --target sky130-fd-sc-hd --systemverilog build/Top.sv
zlang source.zhl --top Top --verify
zlang sim source.zhl --top Top --clock clk --cycles 100
zlang sim source.zhl --top Top --logic-state --clock clk --cycles 100
zlang sim source.zhl --top Top --strict-uninitialized --clock clk --cycles 100
```

```
zlang verify build/verify --mode bmc --depth 20 --work-dir build/verify-work
zlang lock update --project zlang.toml
```

In a `--events` JSONL schedule, `"repeat": N` repeats one event lazily for a bounded positive number of steps (at most 1,000,000 expanded events) without a project-specific generator.

Use `--cycles` only for a one-clock top that does not need an explicit reset sequence. Resettable or multi-clock designs should use JSONL so pin levels, inputs and coincident edges are reproducible:

```
{"reset":{"rst":true},"edges":["clk"]}
{"reset":{"rst":false},"set":{"enable":1},"edges":["clk"]}
{"set":{"enable":0},"edges":["clk"],"repeat":4}
```

Run it with `--events events.jsonl`; add `--trace build/run.fst` and repeated `--trace-signal NAME` options for a selective waveform. Add `--compare-with verilator` or `--compare-with iverilog` to replay the same events against generated Direct-SV. Any mismatch, unresolved RTL output, tool failure, or timeout is an error, not a warning.

Ordinary native simulation retains its deterministic two-state zero seed. `--logic-state` enables per-bit `0`/`1`/`U`/`X`: unreset state starts as `U`, an accepted unresolved write becomes `X`, and controlling values such as `U | 1` still resolve exactly. `--initial-reg` overrides selected initial registers and `--trace-signal` selects waveform signals. Prefer `.fst` for native interactive traces; `.vcd` remains supported for interchange. Hidden `__zlang_meta/*_u_mask` traces distinguish `U` from the formats' shared `x` spelling. `--strict-uninitialized` implies this mode and rejects observed `U` or `X`. Neither mode models `Z`, electrical resolution, or Verilog event regions. CLI FST/VCD output is an event-index visualization: physical clock/reset levels match each compiled domain, but the JSONL schedule does not define analog time.